

第一篇 AI Agent 开发报告

从 `prompt engineer` 到 `harness engineer`

1.1 汇报背景：为什么今天要重新理解 AI Agent

今天讨论 AI Agent，不能只把它理解成“会聊天的大模型”，因为在真实开发里，用户要的不是一段回答，而是一个能持续推进任务、调用工具、读取环境、根据结果修正行为的系统。Anthropic 在《Building Effective AI Agents》中总结了大量落地经验后指出，真正效果好的 Agent，通常不是靠复杂框架堆出来的，而是靠一组简单、可组合的工程模式把模型能力组织起来。OpenAI 在 2026 年也把这种围绕模型构建完整执行系统的实践明确称为 **Harness Engineering**。

因此，AI Agent 开发的核心问题已经不再是“模型够不够聪明”，而是“如何围绕模型搭建一套可靠的工程外壳”。这套外壳需要解决指令表达、上下文组织、工具执行、任务编排、结果验证、安全控制和长期约束等一整套问题。

1.2 第一阶段：从 LLM 出发，先理解模型本体的边界

大语言模型本质上是一个推理与生成服务。OpenAI 官方把 Prompt Engineering 定义为：为模型编写有效指令，使其稳定地产生符合要求的内容。这句话背后意味着一个前提：**模型本体并不天然等于“可靠执行器”**，它首先是一个根据输入生成输出的模型，因此输入如何组织，直接决定输出质量。

也正因为如此，当输入宽泛、目标模糊、格式不清时，模型往往会出现输出漂移、格式不稳定、前后不一致、甚至看似合理但并不可靠的内容。Prompt Engineering 的价值，就是通过角色、目标、约束、格式、示例等方式，把任务表达得更清楚，从而提升单轮生成的稳定性和可控性。

所以在 AI Agent 演进链条中，**Prompt Engineering 是第一步**。它解决的是“怎么把问题说清楚”，让模型更像一个受控的推理模块，而不是一个自由发散的文本生成器。

1.3 第二阶段：从 Prompt Engineering 走向 Context Engineering

当任务从单轮问答变成多轮开发、调试、搜索、改代码时，只优化 prompt 已经不够了。Anthropic 在《Effective context engineering for AI agents》中把这个问题讲得很明确：在 Agent 系统里，真正需要管理的不是一段 prompt，而是**模型在当前时刻能够看到的全部 token 集合**，其中包括系统指令、历史消息、工具说明、外部数据、MCP 连接结果、规则文件和中间状态。

这就引出了 **Context Engineering**。它关注的不是“这句话怎么写”，而是“这一轮到底该给模型看什么”。换句话说，Prompt Engineering 解决的是指令表达问题，Context Engineering 解决的是上下文供给问题。只要上下文窗口有限，这个问题就一定存在，因为模型不可能永远保留所有历史、所有规则和所有环境信息。

在工程实践里，Context Engineering 通常可以拆成三步：**召回、压缩、组装**。召回是从历史记录、知识库、代码仓库、工具反馈、外部服务中取回当前任务真正需要的信息；压缩是对长内容做总结、筛选和结构化，减少 token 消耗；组装则是按优先级把这些信息重新排列进上下文，让当前最关键的约束、目标和状态处在最容易被模型关注的位置。Anthropic 对 context engineering 的讨论，本质上就是在强调这类动态装配能力。

因此，在今天的 AI 工具竞争中，很多能力差异并不只是模型差异，而是**上下文工程差异**。同一个模型，接入不同的召回策略、记忆机制、规则注入方式和上下文排序逻辑，最终表现会非常不同。

1.4 第三阶段：模型为什么必须拥有执行层

即使上下文组织得很好，模型仍然有一个天然限制：**它本身没有现实执行能力**。它不会真的读文件、不会真的改代码、不会真的跑测试、不会真的联网查资料，除非系统给它接入工具。Anthropic 对 Claude Code 的定义非常直接：这是一个会读代码库、编辑文件、运行命令并接入开发工具的 agentic coding 工具；OpenAI 对 Codex 的定义同样强调，它不只是写代码，还能理解代码库并适配项目结构与约定。

所以，一个 Agent 要从“会说”变成“会做”，必须增加**执行层**。执行层通常包括 Bash 沙箱、文件系统访问、代码运行环境、测试命令、浏览器操作、外部 API、数据库，以及更标准化的工具连接方式。Anthropic 在发布 MCP 时把它定义为一种开放标准，用来在 AI 应用和外部数据源、工具之间建立安全的双向连接；MCP 官方文档进一步把它描述为让 AI 应用连接数据源、工具和工作流的通用接口。

一旦执行层存在，模型就不再只是“生成答案”，而是可以发起真实动作：读取工程文件、执行测试、调用接口、查询外部信息、观察结果、再继续下一步。这就是 Agent 和传统聊天模型之间最核心的分水岭之一。

1.5 第四阶段：从工具调用到 ReAct 循环

模型拥有工具之后，还需要一种工作方式，把“思考”和“行动”连接起来。ReAct 论文给出的经典范式是：让模型以交错方式生成 reasoning traces 和 task-specific actions，也就是在推理与动作之间形成循环。论文指出，这种方式的价值在于：推理帮助模型规划和更新动作，动作又帮助模型从环境中获取新的信息，反过来修正后续推理。

这意味着，Agent 的核心不只是“能调用工具”，而是形成一个闭环：**思考 → 行动 → 观察 → 更新 → 再思考**。只要工具结果被重新写入上下文，这个循环就会持续推进任务。对于编码场景来说，这个循环往往体现为：先理解需求，再读代码，再改文件，再跑测试，再根据报错修正实现。

所以，从方法论上说，ReAct 为 AI Agent 提供了最重要的行为基础：Agent 不是一次性回答，而是持续迭代。它能做自动纠错、自动测试、自动补救，本质上都来自这个循环。

1.6 第五阶段：为什么还需要“记忆层”

当任务变长，循环次数变多，上下文必然膨胀。即使有 Context Engineering，也无法把所有历史都永久保留在当前窗口里。Anthropic 在 Claude Code 文档里明确写到：每个 Claude Code 会话都从一个新的 context window 开始，而跨会话知识依赖两种机制携带——**CLAUDE.md** 文件和 auto memory。也就是说，长期规则和关键偏好必须被外置保存，而不能完全依赖会话内记忆。

OpenAI 在 Codex 里也提供了类似机制。官方文档说明，Codex 会在开始工作前读取 `AGENTS.md` 文件，并支持全局指导和项目级覆盖，从而让代理在进入仓库时就继承一致的规则和预期。Best practices 文档进一步把 `AGENTS.md` 描述为适合沉淀团队约定和仓库工作方式的地方。

因此，AI Agent 中的“记忆层”并不只是聊天历史，更重要的是**长期稳定注入的规则、偏好、流程说明、项目知识和环境约定**。这类内容的价值在于：即使当前轮次的上下文被裁剪，系统仍然能通过这些外置文件把关键约束重新注入回来。

1.7 第六阶段：规则文件为什么不能无限膨胀

规则文件解决了长期约束问题，但它也带来新的问题：**规则一多，就会挤占上下文**。OpenAI 在 Codex 的相关文档里一方面鼓励使用 `AGENTS.md` 固化工作方式，另一方面也强调它应该是高质量、可复用的指导，而不是无边界膨胀的长文档。Harness Engineering 一文同样强调，仓库知识需要清晰、可读、可被代理利用，否则代理会在海量说明里迷失重点。

为了解决这个问题，现代 Agent 开始采用**按需加载**的结构化知识组织方式。OpenAI 的 Agent Skills 文档把 skill 定义为：一种将 instructions、resources 和可选 scripts 打包在一起的任务能力单元，用来让 Codex 可靠地遵循某类工作流。也就是说，不是把所有规范一次性塞给模型，而是把规范拆成一个个面向任务的能力包，在需要时再调入。

这一步非常关键，因为它意味着记忆层开始从“静态大文档”升级成“**规则文件 + 技能路由 + 按需装配**”的体系。这样既能保留长期约束，又能减少无关上下文对当前任务的污染。

1.8 第七阶段：从记忆管理走向任务编排

即使有了记忆层和执行层，一个复杂任务仍然可能跑偏，因为模型未必天然会把大任务拆得足够清楚。Anthropic 在《Building Effective AI Agents》中强调，成功的 Agent 系统通常依赖清晰、可组合的流程，而不是让模型在没有约束的情况下自由长跑。

这就引出了**编排层**。编排层解决的是：如何把复杂任务拆成一系列更稳定、更可验证的小步骤，再让代理逐步完成。对于 coding agent 来说，这些步骤通常会表现为：理解需求、确认规范、规划方案、实现接口、补充测试、验证结果、整理交付。这样的流程不是为了模型“更聪明”，而是为了让它**更不容易失控**。

在更复杂的系统中，编排层还会进一步演化成多代理协作。Claude Code 的 subagents 文档明确指出，子代理的用途包括任务专门化、上下文隔离和并行处理；Anthropic 也公开介绍过多代理研究系统如何用多个 Claude 协作探索复杂问题。也就是说，编排层不仅是“列任务清单”，还是“把不同任务交给不同上下文去完成”。

1.9 第八阶段：反馈层决定 Agent 能否真正闭环

只有执行，没有反馈，Agent 很快就会偏航。OpenAI 在 Codex 的安全与沙箱文档里把 sandbox 和 approval 明确拆开：sandbox 决定技术边界，approval 决定什么时候必须停下来请求许可。Anthropic 关于 agent evals 的文章则指出，Agent 的评估通常要结合代码规则、模型判断和人工检查。

这说明，完整的反馈层至少有三类能力。第一类是**结果反馈**，比如测试是否通过、接口是否返回正确、页面是否能运行；第二类是**过程反馈**，比如工具调用是否报错、命令是否失败、权限是否不足；第三类是**安全反馈**，比如某个动作是否越过边界、是否需要人工审批。只有这些反馈被重新写回系统，Agent 才能形成真正可控的闭环。

因此，AI Agent 的“自动纠错”并不是魔法，而是因为系统提供了测试、评估、审批和环境边界，让模型在失败之后还能继续收敛。

1.10 第九阶段：Harness Engineering 作为总装层

当前沿工具把记忆层、编排层、执行层和反馈层都接起来之后，就形成了更完整的工程外壳。OpenAI 在《Harness engineering》中直接用这个词来描述 agent-first 软件开发中的工程实践；Anthropic 在关于 long-running agents 的文章里也多次使用 harness 这一概念，强调长时程任务之所以难，不是因为模型不能输出，而是因为跨多个 context windows 持续稳定推进任务，需要强约束的外部结构。

所以，Harness Engineering 可以理解为围绕大模型构建完整 Agent 系统的工程学。它不是单指 prompt，也不是单指工具调用，而是把上下文管理、长期规则、技能组织、任务编排、执行环境、测试验证、安全审批和记忆机制整合起来，让模型能够在真实世界里可靠工作。

从这个视角看，“AI = 大模型 + Harness”是一种非常适合工程落地的理解方式：模型提供推理能力，Harness 提供落地能力。没有 Harness，模型很难稳定完成长链路任务；没有模型，Harness 也只是一堆静态自动化脚本。

1.11 第十阶段：在 Coding Agent 中，这套体系具体长什么样

在 coding agent 场景下，这个体系已经有很清晰的落地形态。长期规则可以放进 `AGENTS.md` 或 `CLAUDE.md`；项目能力可以抽成 skills；外部系统通过 MCP 接入；高风险动作受 sandbox 和 approval 控制；复杂任务通过子代理或多代理分工；测试和 eval 负责在每轮之后给出反馈；上下文工程负责决定当前轮究竟加载哪些规则、哪些文件、哪些历史和哪些工具结果。

这也是为什么今天很多 AI 编程工具看上去都在“写代码”，但实际能力差异很大。真正拉开差距的，不只是底层模型参数规模，而是它们的 Harness 是否成熟：是否会自动读规则、是否会按需加载技能、是否能安全调用工具、是否能组织长时程任务、是否能在失败后继续修复。

1.12 第十一阶段：Spec-Driven Development 为什么会变重要

当 Agent 可以直接写代码后，开发方式也开始变化。GitHub 在发布 Spec Kit 时提出了 **Spec-Driven Development**，并把它描述为一种结构化流程：先明确 spec，再推进计划、任务和实现，而且这套流程可以接入 GitHub Copilot、Claude Code、Gemini CLI 等 coding agent。GitHub 博客和 Spec Kit 仓库都在强调同一件事：在 agent 时代，规范不再只是开发前的文档，而是可以直接驱动实现的工作入口。

这意味着，开发重点正在从“直接写代码”部分前移到“先把需求、边界、验收条件和实现约束写清楚”。Spec 在这里扮演的是编排层和记忆层之间的桥梁：它既约束了任务目标，也给 Agent 后续规划、执行和验证提供了标准。

因此，在 Agent 开发里，程序员的工作并没有消失，而是在重心上发生了变化：除了写实现本身，还要写规则、写规范、写 skills、写 eval、写测试、写审批边界、写工具协议。也就是说，工程师开始同时扮演系统设计者、约束制定者、流程编排者和质量守门人。

1.13 第十二阶段：AI Agent 的完整分层模型

综合以上内容，可以把一个现代 AI Agent 系统概括为五层。

第一层是**模型层**，负责理解、推理、生成和规划。第二层是**上下文层**，负责 prompt、历史、规则、记忆、检索和工具说明的动态装配。第三层是**执行层**，负责 bash、文件系统、MCP、API 和外部环境交互。第四层是**编排层**，负责计划、拆解、路由、子代理协作和流程推进。第五层是**反馈与控制层**，负责测试、eval、安全边界、审批和异常恢复。这个分层模型，和 Anthropic、OpenAI 近两年的 Agent 实践方向是高度一致的。

如果用一句话概括：Prompt Engineering 让模型更会听话，Context Engineering 让模型每轮都看对信息，ReAct 让模型边想边做，Harness Engineering 则把这一切封装成一个可长期运行、可安全控制、可持续交付的工程系统。

1.14 第十三阶段：落地建议：从轻到重，如何建设自己的 Agent 系统

如果团队刚开始落地，最轻量的做法，是先建立项目级规则文件，把代码规范、常用命令、测试要求、目录约定和禁止事项固化到 **AGENTS.md** 或 **CLAUDE.md** 中，让代理在进入仓库时有统一行为基线。OpenAI 和 Anthropic 的文档都已经把这类文件当成 coding agent 的基础设施。

第二步，可以把重复工作流沉淀为 skills，把“如何初始化模块”“如何补测试”“如何做代码审查”“如何接入接口”这类流程拆成可复用能力包，避免每次都临时拼接 prompt。OpenAI 的 Skills 文档就是为这件事设计的。

第三步，再引入 spec 驱动流程，把需求澄清、计划生成、任务分解、实现、验证连接成固定路径。GitHub 的 Spec Kit 给出的正是这样一套 agent 友好的流程化思路。

第四步，补齐评估和控制层，包括测试命令、静态检查、回归脚本、sandbox 边界和 approval 策略。没有这一步，Agent 只能算“能执行”；完成这一步，Agent 才开始接近“可托付”。

1.15 第十四阶段：总结

AI Agent 开发并不是在做一个更长的 prompt，而是在构建一个以大模型为核心、以上下文工程为组织方式、以工具系统为执行能力、以编排与反馈为闭环机制、以 Harness Engineering 为总装逻辑的工程系统。模型负责推理，Prompt 负责表达，Context 负责供给，Tools 负责执行，Workflow 负责推进，Feedback 负责纠偏，Harness 则把这些能力组合成真正可落地的生产系统。

所以，理解 AI Agent 的关键，不是只盯着模型本身，而是要看到模型外面那整套逐层包裹的工程结构。未来 AI 开发的核心竞争力，也会越来越体现在这套结构是否清晰、是否可控、是否可复用、是否经得起长链路任务和真实业务场景的考验。

第二篇 AI-DLC

1.1 先看结论：AI-DLC 是什么

AI-DLC，通常指 **AI-Driven Development Life Cycle**，是 AWS 在 2025 年公开提出的一套 **AI-native 软件研发生命周期方法**。它不是某一个 IDE、某一个模型、某一套插件，而是一种把 AI 放到研发流程中心、由 AI 主动推进工作、但由人保留关键判断权的方法论。

核心要点：AWS 官方概括为“AI 负责生成计划、提出澄清问题、执行方案；人类负责业务判断、关键决策和最终验证”，开源仓库将其定义为“根据项目复杂度自适应、同时保持质量和人工控制的软件开发工作流”。

开源仓库也把它定义为一种会根据项目复杂度自适应、同时保持质量和人工控制的软件开发工作流。

1.2 AI-DLC 的概念解读

1.2.1 为什么会提出 AI-DLC

AI-DLC 的提出，直接来自对现有 AI 开发方式的不满。AWS 在首篇方法论文章里把当前常见做法归纳成两类：一类是 **AI-assisted development**，也就是 AI 只辅助某些局部任务，比如补全文档、生成代码片段、写测试；另一类是 **AI-autonomous development**，也就是把整套应用几乎完全交给 AI 自动生成。AWS 的判断是，这两类方式在速度和软件质量上都不够理想。

与此同时，传统 SDLC 又主要是为“人类驱动、周期较长、会议和流程较重”的开发方式设计的，因此只是把 AI 硬塞进旧流程里，既限制了 AI 的潜力，也保留了旧流程的低效。

AWS 在后续开源 AI-DLC 工作流时，又把问题说得更工程化了一层：团队在落地 AI 时，经常遇到三类系统性问题。第一是 **one-size-fits-all workflows**，也就是所有项目都被迫走同一条流程；第二是流程深度不灵活，要么过度设计，要么不够严谨；第三是工具过度自动化，把本来应该由人负责的验证和监督挤掉了。

核心要点：AI-DLC 的核心目标是让研发流程变得 **自适应、分层次、并且保留人工决策权**，解决现有 AI 开发方式的效率与质量痛点，适配 AI 原生的工作特性。

1.2.2 AI-DLC 的核心思想

AI-DLC 的核心不是“AI 替代工程师”，而是 **AI 发起并推动执行，人类持续做关键校准**。AWS 官方把这个新心智模型写得非常明确：AI 先创建计划，再通过提问补足上下文，只有在人类验证之后才继续实现；这个模式会在每一个 SDLC 活动中反复出现。

核心要点：AI-DLC 不是“一次性把需求丢给 AI”，而是将 AI 作为流程主动推进者，人类作为关键节点控制者，形成“AI 推进+人类校准”的循环模式。

在这个模型下，AI-DLC 被划分为三个总阶段：**Inception**、**Construction**、**Operations**。AWS 官方定义中，Inception 负责把业务意图变成需求、故事和工作单元；Construction 负责把这些已验证的上下文转成架构、代码和测试；Operations 则负责把前面沉淀下来的上下文继续用于基础设施、部署和运维。每一阶段都会为下一阶段提供更丰富的上下文，而且这些计划、需求、设计工件会被持久化到代码仓库中，便于跨会话延续。

1.2.3 AI-DLC 和传统 SDLC 的区别

如果把 AI-DLC 和传统 SDLC 放在一起看，最本质的区别不在“阶段名称”，而在 **谁主导流程、上下文如何传递、以及验证何时发生**。传统 SDLC 更像是人驱动的串行流程，AI 只是附加助手；AI-DLC 则把 AI 放到流程中心，让它主动提问、主动整理方案、主动推进实施，但在关键节点停下来等人确认。

AWS 还明确提出，AI-DLC 会引入新的节奏和术语：传统的 sprint 被更短、更密集的 **bolts** 替代，epic 则被 **units of work** 替代，强调的是更快的迭代和更直接的交付。

换成更工程化的说法，AI-DLC 可以看成是对 SDLC 的一次 **AI-native 重写**。它不是把 AI 加在 SDLC 上，而是把 SDLC 改造成“AI 提出、AI 落实、人来校准”的模式。与此同时，它又不是完全放权给 AI，因为它明确要求每个关键阶段都经过人类确认，并且通过审计记录和阶段工件把全过程留痕。

1.2.4 AI-DLC 的适应性特点

AI-DLC 还有一个很重要的特点，就是 **不是所有任务都走满全流程**。AWS 在 Amazon Q 的落地文章里明确说明，AI-DLC 会先分析请求、代码库和复杂度，再决定哪些阶段真正需要执行：简单的 bug fix 可能直接跳到代码生成；复杂性则需要先经历需求分析、架构设计和详细测试。

核心要点：AI-DLC 具备自适应智能，会根据任务复杂度动态选择执行阶段，只保留对当前任务有价值的流程，避免冗余，提升效率。

开源仓库 README 也把这一点称为 **adaptive intelligence**：只执行对当前任务有价值的阶段。

1.3 与 AI-DLC 相关联的核心概念

AI-DLC 不是孤立存在的。它在落地时会和一系列概念互相配合。最值得梳理的是：**Spec**、**MCP**、项目级规则文件、**Skills/Workflows/Hooks**，以及上下文记忆机制。

1.3.1 Spec：把“做什么”固定下来

Spec，尤其是 **Spec-Driven Development**，和 AI-DLC 的关系非常紧密，但两者不是同一件事。GitHub 的 Spec Kit 官方文档把 Spec-Driven Development 定义为：让 specification 从过去“写完就丢”的辅助文档，变成 **可以直接驱动实现的可执行工件**。官方还强调，spec 不再只是指导实现，而是成为 implementation 的 **source of truth**。

GitHub 博客进一步把这套流程拆成 **Specify**、**Plan**、**Tasks**、**Implement** 四步。

从 AI-DLC 的角度看，Spec 最适合承载的是 **Inception** 和 **Construction** 之间的结构化中间产物。比如需求、验收标准、设计约束、任务拆解，本来就是 AI-DLC 里需要不断生成、验证和持久化的东西。Kiro 的原生 specs 就很典型：它把 spec 固定成 **requirements.md**、**design.md**、**tasks.md** 三类文件，用来承载需求、设计和可追踪任务。

但也要看到，**AI-DLC 比 Spec 更大**。Spec 更偏向“把目标和约束写清楚”，而 AI-DLC 关心的是“从需求到实现到部署的整条流程怎么由 AI 驱动”。这一点在 AWS 的 Kiro 接入方式里也能看出来：开源 AI-DLC 仓库虽然支持 Kiro，但官方建议是通过 **steering files + Vibe mode** 运行 AI-DLC，而不是直接依赖 Kiro 的 native spec mode。这说明在 AWS 的实践里，Spec 是 AI-DLC 的重要相关能力，但不是唯一入口。

核心要点：Spec 负责明确“做什么”，是 AI-DLC 中连接需求与实现的关键结构化载体，但 AI-DLC 覆盖范围更广，聚焦整条研发流程的 AI 驱动，Spec 只是其重要组成部分而非唯一入口。

1.3.2 MCP：把“能做什么”接进来

MCP (Model Context Protocol) 是 AI-DLC 非常关键的另一条支撑线。Anthropic 对 MCP 的定义是：一种开放标准，用来在数据源和 AI 工具之间建立安全的双向连接。GitHub 对 MCP 的定义也类似：它规定了应用如何把上下文分享给 LLM，并以标准化方式把 AI 连接到不同的数据源和工具上。

对 AI-DLC 来说，MCP 的价值不在“概念很新”，而在它承担了 **执行层和上下文接入层** 的角色。AI-DLC 既要让 AI 问问题，也要让 AI 读代码、查系统、跑工具、连外部服务。如果没有 MCP 或等价机制，这些工具接入只能一对一硬编码，无法形成统一可复用的能力层。

Claude Code、GitHub Copilot、Kiro、Cline 都把 MCP 放在自己扩展能力的重要位置：Claude Code 明确支持通过 MCP 连接数据库、API 和外部工具；Copilot 官方把 MCP 定义成扩展 Copilot 能力的协议；Kiro 也把 steering 与 MCP 结合，作为让 IDE 理解私有库和私有 DSL 的方法。

所以，从 AI-DLC 的视角看，**Spec 负责约束目标，MCP 负责接入能力**。前者解决“做什么”，后者解决“凭什么做”。

核心要点：MCP 承担 AI-DLC 的执行层和上下文接入层角色，解决“凭什么做”的问题，通过标准化连接，让 AI 能够接入外部工具、数据源，为流程执行提供能力支撑。

1.3.3 项目级规则文件：把“怎么做”固化下来

AI-DLC 的开源工作流有一个非常鲜明的特点：它不是只靠一条 prompt，而是靠 **核心规则文件 + 详细阶段规则** 来驱动。AWS 在 Amazon Q 的实操文章里专门解释过：一个核心 **core-workflow.md** 会常驻进入上下文，而更细的阶段规则文件则按需动态加载，这样既能保持流程框架稳定，又能节省上下文窗口。

不同工具承载这套规则的方式不同，但本质是一致的：

- Kiro 用的是 **steering files**，并且有 **product.md**、**tech.md**、**structure.md** 这三类基础 steering 文件，默认参与每次交互；它也支持 **AGENTS.md**，以及 always、fileMatch、manual 三种 inclusion 模式。
- Cline 用的是 **Rules**，可以常驻，也可以按路径条件加载；
- Claude Code 用的是 **CLAUDE.md + auto memory**，两者都会在会话开始时载入；
- Copilot 则用 **.github/copilot-instructions.md** 和 **.github/instructions/*.instructions.md** 承载仓库级和路径级指令。

AWS 的 AI-DLC 仓库也正是沿着这条线提供接入方式：Kiro 放进 **.kiro/steering/**，Cline 放进 **.clinerules/** 或 **AGENTS.md**，Claude Code 放进 **CLAUDE.md**，Copilot 放进 **.github/copilot-instructions.md**。

核心要点：项目级规则文件是 AI-DLC 方法论的核心载体，而非辅助说明，它固化“怎么做”的标准，保证流程一致性，避免 AI-DLC 退化为普通 agent 聊天。

1.3.4 Skills、Workflows、Hooks、Subagents：把流程变成可执行模块

当规则文件解决了“长期约束”之后，下一步就是把可重复的经验进一步做成 **模块化流程能力**。这就是 Skills、Workflows、Hooks、Subagents 的价值。

各类工具的具体实现的：

- Kiro 的 **skills** 采用 progressive disclosure：启动时只加载 skill 的名称和描述，请求匹配时才加载完整说明，再按需读取脚本和参考资料；**hooks** 则能在保存文件、工具调用前后、spec 任务执行前后等事件自动触发 shell 命令或 agent prompt。
- Cline 的官方定位也很清楚：它有五套定制系统—Rules、Skills、Workflows、Hooks、**.clineignore**；其中 skills 只在相关请求时加载，workflows 用 Markdown 文件把多步任务封成 **/workflow.md** 命令，hooks 用于在关键事件注入自定义逻辑。
- Claude Code 则提供了 **skills、hooks、subagents**：skills 通过 **SKILL.md** 扩展能力，hooks 可以在生命周期关键点自动运行命令或注入上下文，subagents 内置 Explore、Plan、General-purpose 等不同角色。
- GitHub Copilot 的对应物是 **custom agents**，官方文档说明它们通过 agent profile 定义 prompts、tools 和 MCP servers。

1.3.5 记忆与上下文管理：把流程跨会话接起来

AI-DLC 强调全过程留痕和跨阶段连续性，所以它天然依赖 **上下文持久化**。AWS 官方说得很清楚：AI-DLC 会把计划、需求和设计工件保存在仓库里，以便跨多个 session 继续工作。

不同工具在这方面的实现也不一样：

- Claude Code 有 **CLAUDE.md** 和 **auto memory** 两套记忆系统，都会在每次会话开始时加载；
- Cline 提供 **Memory Bank**，把项目背景、当前进度、架构模式、技术上下文等拆成结构化 Markdown 文件保存，还配合 **Checkpoints** 和 **Auto Compact** 管理上下文和回滚；
- Kiro 更依赖 steering、specs 和规则文件的组合来维持长期上下文；
- Copilot 则更多依赖仓库内的 instructions、custom agents 和 GitHub 平台环境。

核心要点：上下文持久化是 AI-DLC 实现“生命周期”的基础，而非锦上添花，它保证跨会话、跨阶段的连续性，避免流程断裂，让 AI 能够持续推进研发工作。

1.4 与 AI-DLC 相关的工具：Kiro、Cline、Claude Code、Copilot 到底有什么区别

这四类工具都能承载 AI-DLC，但它们的重心不同。比较它们时，最有价值的角度不是“谁更强”，而是 **谁更像哪一种 AI-DLC 载体**。

1.4.1 Kiro: 更偏“Spec / Steering 原生”的 AI IDE

Kiro 官方把自己定义为 **agentic IDE**，核心能力是 **specs**、**steering**、**hooks**。官网首页直接把“spec-driven development”和“advanced steering”摆在最前面；文档里又把 **specs** 定义成把高层想法变成结构化实现计划的工件，并且固定生成 **requirements.md**、**design.md**、**tasks.md** 三类文件。

与此同时，Kiro 的 **steering** 机制还提供 **product**、**tech**、**structure** 三份基础上下文，并支持 **AGENTS.md** 与按模式加载。

从 AI-DLC 适配度看，Kiro 的优势在于它很自然地承接 **Inception 阶段的结构化产物**。如果团队希望在 IDE 内直接把需求、设计、任务分解和长期规则放在同一套界面里，Kiro 会很顺手。AWS 的开源 AI-DLC 也给出了 Kiro 的专门接入方式：把 AI-DLC 的核心规则放进 **.kiro/steering/**，并在 Kiro 里用 **Vibe mode** 运行该流程。

1.4.2 Cline: 更偏“可组合、可控、可回滚”的执行代理

Cline 官方文档最突出的特点，是把定制能力拆得非常细：**Rules**、**Skills**、**Workflows**、**Hooks**、**.clineignore** 五套系统，再加上 **Plan & Act**、**Memory Bank**、**Checkpoints**、**Auto Compact**。Plan mode 明确只允许读代码和做策略讨论，不允许改文件或跑命令；Checkpoints 用 shadow Git repo 为每一步保存快照；Memory Bank 则用结构化 Markdown 维护跨会话知识。

从 AI-DLC 的角度看，Cline 最突出的长处不是 **spec 原生**，而是 **执行层和控制层做得很细**。它很适合承载那种需要大量本地文件操作、频繁命令执行、强上下文清理、强回滚能力的 AI-DLC 落地方式。AWS 也正是基于这一点，让 AI-DLC 在 Cline 中通过 **.clinerules/** 或 **AGENTS.md** 来实现。

核心定位：Cline 更像高可定制的 **agent runtime / 执行外壳**，长处在于执行层和控制层的精细化设计，适合需要大量本地文件操作、强过程控制和回滚能力的 AI-DLC 落地场景。

1.4.3 Claude Code: 更偏“内建 agent 架构完整”的 coding agent

Claude Code 官方定位是：一个能读代码库、改文件、跑命令、连开发工具的 **agentic coding tool**，可运行在 **terminal**、**IDE**、**desktop app** 和 **browser**。它内建了很多 **agent 级能力**：**CLAUDE.md** 和 **auto memory** 用来保存持久上下文；**hooks** 在生命周期关键点自动运行；**skills** 用 **SKILL.md** 扩展能力；**subagents** 则提供 **Explore**、**Plan**、**General-purpose** 等角色化代理。它还直接支持通过 **MCP** 连接外部工具。

从 AI-DLC 的映射看，Claude Code 更像一个 **能力比较均衡的通用 agent 平台**：既有长期记忆入口，又有技能和钩子，又有子代理和 **MCP**，所以很适合用来承接 AI-DLC 这种“既要流程、又要执行、还要扩展”的方法。AWS 的开源仓库在 Claude Code 上的实现方式也很直接：把 AI-DLC 的核心规则放进 **CLAUDE.md**，把阶段细节放到 **.aidlc-rule-details/**。

1.4.4 GitHub Copilot: 更偏“GitHub 平台内 workflow”的云端代理

Copilot 的官方文档现在已经不只是“补全代码”了。GitHub 对 **Copilot cloud agent** 的描述是：所有编码和迭代都可以发生在 GitHub 上，用户可以让 Copilot 先研究仓库、创建计划、在分支上改代码，再决定是否开 PR；它还会自动建分支、写 commit message、push 代码。

与此同时，Copilot 还支持 **.github/copilot-instructions.md**、路径级 **instructions**、**custom agents** 和 **MCP**。**custom agents** 通过 **agent profile** 定义 **prompts**、**tools** 和 **MCP servers**。

这意味着 Copilot 的最大特色，在于它和 **GitHub 原生工作流** 结合得最紧。它更适合那种 issue、branch、PR、review、GitHub Actions 都已经是团队主流程的场景。AWS 的 AI-DLC 仓库也正是借助这一点，把核心 workflow 放进 `.github/copilot-instructions.md`，从而把 AI-DLC 接入 Copilot。

1.4.5 四类工具核心定位总结

如果把这四类工具放到 AI-DLC 的方法论里，比较清晰的定位是：

- Kiro 更强在 **规格表达和项目上下文显式化**；
- Cline 更强在 **流程自定义、执行控制和回滚**；
- Claude Code 更强在 **agent 组件完整度和平衡性**；
- Copilot 更强在 **GitHub 平台集成和云端工作流闭环**。

这个结论不是某家官方直接说的话，而是基于它们各自文档和 AWS 的 AI-DLC 接入方式做出的工程归纳。

1.5 AI-DLC 在实际软件开发生命周期中的落地过程

下面把 AI-DLC 真正落回“软件是怎么一步一步做出来的”，按流程拆解为八个关键步骤，明确每一步的核心动作和要求。

1.5.1 第一步：先把 AI-DLC 工作流装进你的工具里

AI-DLC 的开源实现不是“下载一个 CLI 就自动完成”，而是把 **核心工作流规则文件** 和 **阶段详细规则文件** 接入你使用的代理工具。AWS 的仓库已经给出了不同工具的具体放置方式：

- Kiro 放到 `.kiro/steering/`；
- Cline 放到 `.clinerules/` 或 `AGENTS.md`；
- Claude Code 放到 `CLAUDE.md`；
- Copilot 放到 `.github/copilot-instructions.md`。

详细阶段规则则以 `.aidlc-rule-details/` 或工具特定目录形式放在旁边。

核心要点：这一步的核心价值是让 AI 被稳定的工作流规则驱动，而非依赖当前聊天上下文猜测流程，同时通过“核心规则常驻+阶段规则按需加载”，降低上下文消耗。

AWS 在 Amazon Q 的实践文章里还特别解释过，核心 workflow 会长期进入上下文，而详细阶段规则只会按需动态加载，从而降低上下文消耗。

1.5.2 第二步：用一个高层问题陈述启动 AI-DLC

AWS 的实践示例里，启动方式非常简单：在对话里以 **“Using AI-DLC, ...”** 开头输入你的问题陈述，工作流就会被触发。仓库 README 也把这一点写成标准用法：先表达 intent，再由 AI-DLC 自动接管后续流程，提出结构化问题，生成并要求你审核执行计划。

1.5.3 第三步：进入 Inception，相当于“先把问题真正说清楚”

在 AI-DLC 里，**Inception** 是最关键的起步阶段。开源仓库把它概括成“决定 WHAT to build and WHY”，包括需求分析与验证、必要时的用户故事、应用设计、并行开发单元划分，以及风险和复杂度评估。

在 Amazon Q 的实际演示里，Inception 的核心动作的：

- **Workspace Detection**: AI 会判断当前是 greenfield（新项目）还是 brownfield（已有项目），新项目直接走 Requirements Analysis，已有项目先做 Reverse Engineering 再澄清需求；
- 创建 `aidlc-docs` 目录：生成状态与审计文件，记录过程、支持恢复；
- **Requirements Analysis**: AI 主动提出结构化澄清问题，避免擅自假设，待矛盾、歧义解决后再推进，简单应用可建议跳过 User Stories，支持人工覆盖判断。

核心要点：Inception 阶段的核心不是写需求文档，而是让 AI 与人对目标形成统一理解，为后续 Construction 阶段奠定基础，避免方向跑偏。

1.5.4 第四步：进入 Construction，把已经确认的意图变成代码和测试

Construction 是 AI-DLC 的第二大阶段。仓库把它概括成“决定 HOW to build it”，包括详细组件设计、代码生成、构建配置、测试策略和质量验证。AWS 在方法介绍里则说得更直白：Construction 会基于 Inception 中已经验证过的上下文，让 AI 提出逻辑架构、领域模型、代码方案和测试，再由团队在关键技术点上做实时澄清。

在实际 workflow 中，Construction 并不是一次性“吐出所有代码”。AWS 的实操文章指出，AI-DLC 会根据复杂度决定是否需要额外的功能设计、非功能需求设计、基础设施设计等步骤；但无论如何，**Code Generation** 和 **Build/Test** 都是构造阶段的核心闭环，而且会针对每个 unit of work 循环执行。简单任务可以减少前置设计，复杂任务则会增加设计和测试强度。

核心要点：Construction 与普通“AI 写代码”的根本区别，在于它先验证目标理解，再逐步推进实现，通过 Build/Test 接收反馈，而非盲目产出代码。

1.5.5 第五步：进入 Operations，把交付和运行纳入方法论

AI-DLC 的第三阶段是 **Operations**。AWS 在方法总览里明确把它定义为“部署和监控”，包括基础设施自动化、监控与可观测性、生产就绪性验证。

不过要注意一个现实细节：在 AWS 开源仓库当前的 README 里，Operations 仍被标注为 **Deployment and monitoring (future)**。这说明 Operations 在方法框架里已经被纳入，但在开源 workflow 的成熟度上仍在继续演进。也就是说，AI-DLC 不是只停留在“写代码”，但它在公开实现里，对运维阶段的支持还没有像 Inception 和 Construction 那么完整。

核心要点：Operations 是 AI-DLC 生命周期的的重要组成部分，虽已纳入方法论框架，但当前开源 workflow 中，其成熟度仍在演进，暂未达到 Inception 和 Construction 阶段的完整度。

1.5.6 第六步：全过程都要“人审人批”，而不是最后一次性验收

AI-DLC 最重要的落地原则之一，就是 **人工监督不是终点动作，而是贯穿动作**。AWS 在 AI-DLC 总体介绍里反复强调，AI 会先生成计划、提出澄清问题、等待批准后再实现；在 Amazon Q 实操文里也明确说明，工作流在每个阶段都会要求审阅、批准，并保留审计痕迹。开源仓库的用法说明同样要求你认真审核每个 plan、每个阶段产物和执行方案。

这意味着，AI-DLC 的正确使用方式不是“让 AI 自己跑完，再统一检查”，而是 **阶段性 gating**。你要在需求确认、设计确认、执行计划确认、构建与测试结果确认这些节点不断校准方向。这样做虽然看起来比“全自动”更慢，但它恰恰是 AI-DLC 把速度和质量同时做起来的关键。

核心要点：AI-DLC 采用“阶段性 gating”模式，人工验证贯穿需求确认、设计确认、执行计划确认等关键节点，而非终点一次性验收，这是兼顾速度与质量的关键。

1.5.7 第七步：把工件留在仓库里，让下一个阶段和下一个会话都能接上

AI-DLC 之所以是 lifecycle，而不是一次性 workflow，很大程度上依赖于它会把过程工件保存下来。AWS 说明这些 artifacts 会进入仓库，保证跨会话连续性；在 Amazon Q 示例里，这些工件会进入 `aidlc-docs/`，其中包括状态文件和审计文件。仓库 README 也明确写了：所有 artifacts 都会生成在 `aidlc-docs/` 目录下。

核心要点：过程工件持久化到仓库，是 AI-DLC 实现跨阶段、跨会话连续性的基础，与 Spec、Memory Bank 等共同构成 AI-DLC 的“长期记忆面”，避免流程断裂。

1.5.8 第八步：在真实团队里，先补齐环境和约束，避免 AI 反复猜

AI-DLC 的实战经验还说明了一件很重要的事：如果团队的技术环境、约束、非功能需求没有事先写清楚，AI 就会不停补问，甚至做出会导致返工的假设。AWS 的 `technical-environment-guide.md` 直接说明，这类环境文档的作用就是让代码生成、基础设施设计和非功能需求决策与组织标准保持一致；如果没有它，AI-DLC 阶段要么会提出大量澄清问题，要么会做出错误假设。

核心要点：AI-DLC 虽由 AI 驱动，但需要团队前置明确业务目标、技术边界、合规要求等约束，让 AI 聚焦于流程推进，而非猜测边界，提升效率、减少返工。

1.6 把整套关系压缩成一张方法论地图

把全文收束起来，可以得到这样一张清晰的 AI-DLC 方法论分层关系图，各层级职责明确、相互支撑：

- **AI-DLC**：上位方法论，定义“AI 如何主导研发流程”；
- **Spec / SDD**：结构化工件层，定义“目标、约束和任务如何被写清楚”；
- **MCP**：工具连接层，定义“AI 如何接外部系统和能力”；
- **Steering / Rules / CLAUDE.md / copilot-instructions**：长期规则层，定义“AI 每轮都该遵守什么”；
- **Skills / Workflows / Hooks / Subagents / Custom Agents**：流程执行层，定义“复杂经验如何被模块化复用”；

- **Kiro、Cline、Claude Code、Copilot**：工具承载层，是上述所有能力的具体落地载体，各有偏重。

这个分层理解，和 AWS 的 AI-DLC 工作流仓库、Kiro/Cline/Claude/Copilot 官方文档的组合是吻合的。

1.7 最后的总结

AI-DLC 不是“AI 帮忙写代码”的别名，而是把 AI 放到软件生命周期中心、由 AI 主动推进、由人类持续校准的一套研发方法。它提出的背景，是传统 SDLC 对 AI 不够友好，而单纯的 AI-assisted 或 AI-autonomous 又都难以兼顾速度与质量。

它的关键不只是三个阶段，而是那条反复出现的主线：**AI 先提计划，AI 再问清楚，AI 再执行；人类在关键节点不断验证。**

落到工程实践里，AI-DLC 需要的不只是一个“更会写代码的模型”，而是一整套配套能力：用 Spec 固定目标，用 MCP 接入能力，用 Rules/Steering/Instructions 固化长期约束，用 Skills/Workflows/Hooks/Subagents 组织执行，用仓库工件和记忆机制维持跨阶段连续性。Kiro、Cline、Claude Code、Copilot 的差异，本质上也就是它们各自在这几层里的偏重不同。

第三篇 AI-DLC 示例讲解（Habit Board项目）

本次AI-DLC实践以轻量前端项目为载体，项目名设定为：**Habit Board（习惯打卡板）**。该项目为典型的轻量前端应用，不接入后端，仅基于 **Vue3 + Vite** 开发单页应用（SPA），复杂度适中，可聚焦于AI-DLC流程本身，而非业务难度。Vue官方目前推荐新项目优先使用Vite，**create-vue** 生成的项目默认基于Vite，并可按需勾选Router、Pinia、Vitest、E2E、ESLint、Prettier等功能。

项目核心功能共四类，简洁明确：

1. 新增习惯
2. 每日打卡
3. 查看连续打卡天数
4. 本地持久化保存

1.1 先把 AI-DLC 和 Codex 对齐

AI-DLC 的核心并非“让AI直接写代码”，而是让AI **先建计划、再提澄清问题、再在人工确认后执行**。AWS对AI-DLC的描述明确：其工作模式为 **AI创建计划、主动澄清上下文、仅在人类验证后实施**；整个方法分为 **Inception、Construction、Operations** 三个阶段，且流程会根据项目复杂度自适应——简单改动可跳过不必要阶段，复杂需求则会执行更完整的分析和测试。

Codex方面，官方将Codex CLI定义为可在本地终端运行的coding agent，能够**读取、修改并运行当前目录下的代码**。Codex在工作前会读取 **AGENTS.md** 文件，并支持Skills、Subagents、sandbox和approvals等功能。AWS开源AI-DLC仓库明确说明，AI-DLC可用于**任何支持项目级规则或steering files的coding agent**。

综上可得出结论：**虽然AI-DLC最初并非围绕Codex设计，但完全可以用Codex承载AI-DLC流程。**

核心要点：AI-DLC的核心是“计划-澄清-验证-执行”的循环，Codex的规则读取、代码操作能力，可完美适配AI-DLC的流程要求，成为其落地载体。

1.2 Vue3 + Codex 的起步方式

实践第一步为搭建项目脚手架与安装Codex CLI，均遵循官方标准流程，确保环境合规、可复用。

1.2.1 搭建Vue3 + Vite项目脚手架

Vue官方推荐使用 **create-vue** 生成基于Vite的Vue应用，Vite官方也提供了直接创建Vue模板的命令。需注意，Vite当前文档要求Node.js版本至少为 **20.19+** 或 **22.12+**。

实用起步方式（推荐）：

```
1 npm create vue@latest habit-board
2 cd habit-board
3 npm install
4 npm run dev
```

无提示快速创建方式（直接使用Vite模板）：

```
1 npm create vite@latest habit-board -- --template vue
2 cd habit-board
3 npm install
4 npm run dev
```

上述脚手架命令与启动方式均来自Vue、Vite官方文档，确保兼容性和规范性。

1.2.2 安装Codex CLI

通过npm全局安装Codex CLI，安装完成后启动即可，首次运行需完成登录验证。

```
1 npm i -g @openai/codex
2 codex
```

根据OpenAI官方文档说明，Codex CLI可本地运行，支持读取、修改、运行代码，登录后即可正常使用。

1.3 用 Codex 承载 AI-DLC：先搭“规则层”

AI-DLC要稳定运行，不能依赖单次prompt，需先建立项目级规则层。对Codex而言，规则层的最佳入口是 **AGENTS.md** 文件——官方明确说明：Codex在执行任何工作前，都会先读取 **AGENTS.md**。

因此，Vue3项目落地AI-DLC的第一步，并非让Codex直接编写页面，而是在仓库根目录创建 **AGENTS.md** 文件。该文件的核心作用的：

- 固定项目目标
- 固定技术边界
- 固定开发顺序
- 固定验证标准
- 固定“先问再做”的行为模式

以下为适配Habit Board项目的实用 **AGENTS.md** 示例：

```
1 # Habit Board AI-DLC Rules
2
3 # Project Goal
4 Build a lightweight Vue 3 + Vite SPA for habit tracking with local
  persistence.
5
6 # Stack
```

```

7 - Vue 3
8 - Vite
9 - Composition API
10 - localStorage
11 - No backend in v1
12 - Minimal dependencies
13
14 # AI-DLC Workflow
15 1. Always start with clarification and planning before editing code.
16 2. Create or update docs in `docs/aidlc/` first.
17 3. Implement only one approved task at a time.
18 4. After each code task, run validation commands.
19 5. Summarize changes, risks, and next step after every implementation turn.
20
21 # Definition of Done
22 - App can create, check in, and persist habits
23 - Empty / loading / error-like edge states are handled
24 - `npm run build` passes
25 - Unit tests for core state logic pass
26 - Code stays simple and component boundaries are clear

```

核心要点：**AGENTS.md** 是AI-DLC工作流的长期约束入口，并非补充说明，其作用是规范AI行为，确保流程稳定、不偏离项目目标。

1.4 第一阶段：Inception — 先决定做什么、为什么做

根据AWS对AI-DLC的定义，Inception阶段负责明确 **WHAT to build and WHY**，核心工作包括需求分析、用户故事、应用设计、风险和复杂度评估。该阶段会根据项目复杂度自适应调整——简单项目会简化流程，复杂项目则会展开更细致的分析。

对于Habit Board这个轻量Vue3项目，Inception阶段的核心原则是“不直接写组件”，而是让Codex先完成三件事：

1. 问清需求（澄清歧义，不做假设）
2. 生成结构化规格文档
3. 输出可执行的任务计划

可向Codex下达如下指令，启动Inception阶段：

```

1 先只做 AI-DLC 的 Inception，不要修改代码。
2 请基于当前 Vue3 项目，为 Habit Board 生成：
3 1. 需求澄清问题
4 2. docs/aidlc/requirements.md
5 3. docs/aidlc/design.md
6 4. docs/aidlc/tasks.md
7 如果信息不足，先向我提问，不要假设。

```

该指令完全契合AI-DLC的核心模式：**AI先出计划、先问澄清问题、等人确认后才执行**。AWS在Amazon Q的AI-DLC实践文章中也强调，工作流会先进入Workspace Detection，再进入Requirements Analysis；在需求阶段，规则会刻意要求模型**避免替用户做假设，而是主动提出澄清问题**。

对于本项目，Inception阶段的最终产物应包含：

- **requirements.md**：明确功能范围、非目标、边界条件
- **design.md**：确定组件划分、状态流、数据结构、持久化方式
- **tasks.md**：将开发工作拆分为可分步执行、可验收的小任务

此阶段的人工职责非常明确：**不修改代码，只审核规格文档**。重点关注以下四点：

- 功能范围是否清晰，是否符合项目目标
- v1版本范围是否足够精简，无过度扩展
- 设计方案是否简洁，无过度设计
- 任务拆分是否合理，可逐项验收

核心要点：AI-DLC在前端项目中的第一个关键落地点，是“先把‘做什么’定死，再进入实现阶段”，通过Inception阶段的规格审核，避免后续开发偏离方向、出现返工。

1.5 第二阶段：Construction — 决定怎么实现，并逐步编码

AWS对Construction阶段的定义是明确**HOW to build it**，核心工作包括详细组件设计、代码生成、构建配置、测试策略、质量验证。

对于Habit Board项目，该阶段的核心原则是“不一次性完成所有开发”，而是严格按照 **tasks.md** 中的任务，逐项推进、逐项验证。结合项目特点，合理的任务拆法如下：

1. T1：定义habit数据模型与localStorage持久化服务
2. T2：实现habit列表与新增表单组件
3. T3：实现每日打卡逻辑与连续打卡天数（streak）计算
4. T4：补充边界处理（空状态、重复名称校验、日期边界）
5. T5：补充Vitest单测（核心状态逻辑）
6. T6：构建检查与交付说明生成

Vue官方脚手架支持在创建项目时，直接勾选Vitest、ESLint、Prettier等功能，可将“测试与代码规范”同步纳入Construction阶段，确保开发质量。

可向Codex下达逐任务推进指令，示例如下（以T1为例）：

```
1 现在进入 Construction。
2 只实现 tasks.md 中的 T1:
3 - 定义 Habit 类型/数据结构
4 - 封装 localStorage persistence
5 - 不处理 UI
6 完成后运行测试或最小验证，并总结改动。
```

完成T1后，再下达T2指令，依次推进，示例如下：

```
1 继续 T2:
2 - 实现 HabitForm 和 HabitList
3 - 保持组件职责清晰
4 - 不引入 UI 组件库
5 - 完成后运行可用性验证
```

核心要点：这种逐任务推进的方式，契合AI-DLC的“阶段性gating”原则——人工每轮只批准一个小任务，AI每轮只推进一个清晰目标，既保证开发节奏，又便于及时发现问题、校准方向。

1.6 在 Codex 里，Construction 阶段最适合用的三个能力

Construction阶段是代码实现的核心阶段，合理运用Codex的三个核心能力，可提升开发规范性和效率，更好地承载AI-DLC流程。

1.6.1 AGENTS.md：固定工程边界

AGENTS.md 适合存放项目的长期工程规则，无需频繁修改，确保AI在开发过程中始终遵循统一标准，例如：

- 组件拆分原则（单一职责）
- 状态管理原则（优先Composition API）
- 样式约束（不引入复杂UI库）
- 测试门槛（核心逻辑必须覆盖单测）
- 依赖约束（不引入不必要的第三方依赖）

由于Codex每次工作前都会读取该文件，因此它是承载“项目长期约束”的最佳载体，确保AI开发行为不偏离工程规范。

1.6.2 Skills：封装可复用工作流

根据Codex官方定义，Skill可打包指令、资源和可选脚本，让Codex更稳定地遵循某类工作流；且Skill采用 **progressive disclosure** 机制，仅在命中相关任务时才加载完整的 **SKILL.md**，可有效节省上下文资源。Codex会自动从仓库的 **.agents/skills** 等路径扫描Skill。

针对Habit Board Vue3项目，推荐配置3个仓库级Skill，覆盖组件开发、前端测试、发布检查三大核心场景，目录结构如下：

```
1 .agents/  
2   skills/  
3     vue-component-skill/  
4       SKILL.md  
5     frontend-test-skill/  
6       SKILL.md  
7     release-check-skill/  
8       SKILL.md
```

以 `vue-component-skill/SKILL.md` 为例，可写入如下约束，规范Vue组件开发：

```
1 ---  
2 name: vue-component-skill  
3 description: Use when implementing or refactoring Vue SFC components in this  
4 repository.  
5 ---  
6 1. Prefer Composition API and `核心要点: Skill的核心价值是将团队的前端  
   开发经验，从临时prompt中抽离出来，变成仓库内可复用、可维护的规范，  
   确保AI开发的组件符合团队标准。2.6.3 Subagents: 并行做“查、写、  
   审”Codex官方说明，Subagents适合将复杂任务拆分为多个并行的专门  
   agent，分别处理不同环节，例如代码探索、多步骤功能计划、并行审查等；  
   其内置了 `default`、`worker`、`explorer` 三类agent，可按需调用。对于  
   Habit Board这类轻量项目，Subagents无需全程使用，但在两个场景下价值  
   显著：场景1：并行审查当某一功能模块开发完成后，可启动多个Subagents  
   并行审查，提升问题发现效率，示例指令如下：请并行启动 3 个  
   subagents:  
7 1. 检查 Vue 组件边界是否合理  
8 2. 检查 localStorage 与日期逻辑是否有 bug  
9 3. 检查测试遗漏  
10 等全部完成后汇总问题与修改建议。
```

场景2：先探索再实现

若操作的是已有Vue项目（非全新创建），可先让 `explorer` 类型的Subagents读取现有代码、梳理项目结构，再让 `worker` 类型的Subagents进行开发改动，降低修改风险。

1.7 AI-DLC 在前端实现里的“人机分工”应该怎么划

落地AI-DLC最容易踩的坑，是将其误解为“Codex全自动开发”。实际上，AI-DLC更强调 **AI主动推进，人类把控关键关口**。AWS明确描述：AI会在每个阶段提出问题、编写计划、等待人工审批；人工验证不是最后一次性验收，而是在每个阶段进行“小批准”，确保方向正确。

结合Habit Board Vue3项目，人机分工可明确如下：

人工（开发者）负责：

- 决定v1版本的功能范围，避免过度扩展
- 审批Inception阶段的需求和设计方案
- 拦截过度设计，确保项目轻量化
- 决定是否引入Router、Pinia、E2E等额外功能
- 决定项目的上线方式和交付标准

Codex负责：

- 生成需求、设计、任务等规格草案
- 将开发工作拆分为可分步执行的小任务
- 按人工批准的任务，逐一实现代码
- 运行构建、测试命令，验证代码可用性
- 总结每轮改动的内容、潜在风险和下一步计划
- 生成部署说明、验收清单等交付文档

核心要点：AI-DLC在前端项目中的正确姿势是—AI负责流程推进和具体实现，工程师负责把控方向、界定边界和审批关键节点，二者协同兼顾效率与质量。

1.8 第三阶段：Operations — 轻量项目也要有交付闭环

AWS将AI-DLC的Operations阶段定义为部署和监控；在开源workflow中，该阶段目前仍处于持续演进中，但框架上已包含部署自动化、监控和生产就绪性验证三大核心内容。

对于Habit Board这类轻量Vue3项目，Operations阶段无需复杂配置，但必须形成交付闭环，最小化闭环至少应包含：

- 生产环境构建（生成可部署的静态文件）
- 静态部署说明（明确部署方式和步骤）
- 环境变量说明（若有）
- 简单回滚方式（应对部署异常）
- 基础验收清单（验证部署后功能正常）

根据Vite官方文档，生产打包可直接运行 `vite build` 命令，默认生成适合静态托管服务部署的应用包（dist目录）。

可向Codex下达如下指令，启动Operations阶段：

- 1 现在进入 Operations。
- 2 请完成：
- 3 1. 运行生产构建
- 4 2. 生成 docs/aidlc/deploy.md
- 5 3. 生成 docs/aidlc/release-checklist.md
- 6 4. 说明这个 Vue3 SPA 适合怎样的静态托管方式
- 7 不要引入复杂云服务配置，只做轻量交付。

该阶段完成后，应得到以下产物，确保项目从“本地可跑”升级为“可交付”：

- `dist/`：生产构建后的静态文件目录
- `docs/aidlc/deploy.md`：静态部署说明文档
- `docs/aidlc/release-checklist.md`：发布验收清单

核心要点：Operations阶段的价值是为AI-DLC流程闭环，即使是轻量前端项目，也需通过该阶段确保项目可交付、可部署，避免停留在开发阶段。

1.9 Codex 在 AI-DLC 里的安全使用方式

根据Codex官方安全文档说明，默认情况下，本地agent处于**网络关闭**状态；本地运行时会使用 **os级 sandbox**，通常将写权限限制在当前工作区，同时通过 **approval policy** 控制哪些动作需先请求人工批准。

官方提供的实用建议：若仅需进行需求分析、计划制定等聊天或规划类操作，无需修改代码，可将Codex切换到 **read-only** 模式，进一步提升安全性。

这种安全机制与AI-DLC的阶段控制天然契合，推荐实践如下：

- **Inception阶段**：尽量使用read-only模式，仅进行分析、文档生成，不修改代码
- **Construction阶段**：切换到workspace-write模式，按任务批准代码修改，避免误操作
- **Operations阶段**：允许运行构建命令，谨慎放开网络权限（如无需网络则保持关闭）

核心要点：将Codex的sandbox、approval机制与AI-DLC的阶段控制结合，可实现“先思考、再批准、后执行”的安全流程，避免AI误操作影响项目。

1.10 把整个流程压成一条实际可执行链路

结合上述所有内容，可提炼出一条最实用、可直接落地的Vue3 + Codex + AI-DLC执行链路，共6个步骤，简洁清晰、可复用：

步骤1：脚手架启动

使用 `create-vue` 搭建Vue3 + Vite项目，安装依赖并启动dev server，确保项目基础环境可正常运行。

步骤2：建规则层

在仓库根目录创建 `AGENTS.md` 文件，固定项目目标、技术边界、开发流程和验收标准（Definition of Done），Codex会在执行工作前自动读取该文件。

步骤3：做Inception

让Codex仅产出需求澄清问题、设计草案、任务拆解，不修改业务代码；人工负责审核并批准规格文档，明确“做什么”和“为什么做”。

步骤4：做Construction

按 `tasks.md` 中的任务，逐项推进代码实现，依次完成数据层、组件层、交互层、测试层开发。必要时使用Skills固定Vue组件规范，用Subagents进行并行评审，确保开发质量。

步骤5：做Operations

运行 `vite build` 生成生产包，让Codex生成部署说明（`deploy.md`）和发布清单（`release-checklist.md`），完成项目交付闭环。

步骤6：全程保留docs/aidlc工件

将 `requirements.md`、`design.md`、`tasks.md`、`deploy.md` 等所有阶段工件提交到仓库，确保下一次继续开发时，Codex可基于项目历史和约束继续推进，无需从零开始。这也契合AI-DLC官方强调的“阶段产物留痕、支持连续执行”的要求。

1.11 前端视角下的最终理解

用Codex落地AI-DLC，核心价值不是“让AI写代码更快”，而是将前端项目的开发模式，从传统的“一句需求→直接生成代码”，升级为更规范、更可控的流程：

高层问题 → 规格澄清 → 设计确认 → 小任务实现 → 构建验证 → 交付说明

在这条流程链条中，各组件的作用清晰明确：

- `AGENTS.md`：负责承载项目长期规则，规范AI行为
- `docs/aidlc/*.md`：负责记录各阶段工件，确保流程可追溯
- `.agents/skills`：负责封装可复用的前端开发经验，提升开发规范性
- Subagents：负责并行探索、审查，提升开发效率和质量
- `sandbox + approvals`：负责安全控制和节奏把控，避免误操作

综上，Codex版本的AI-DLC落地，并非复刻AWS专属工具链，而是将AI-DLC的“计划-澄清-验证-执行”方法论，映射到Codex提供的规则、技能、子代理和安全机制上，实现前端项目的规范、高效开发。

1.12 Spec在AI-DLC中的阶段定位及Spec Kit的落地方式

结合AI-DLC的阶段定义与Spec Kit的官方流程，可明确Spec的阶段定位及Spec Kit的落地路径，核心结论先明确：

Spec主要属于AI-DLC的Inception阶段，是Inception阶段的核心工件；而Spec Kit中的 **plan** 和 **tasks** 处于Inception后半段，作为进入Construction阶段的桥梁。

具体拆解如下：

AWS对AI-DLC的定义明确：Inception阶段负责将业务意图转化为详细需求、用户故事和工作单元；Construction阶段则基于Inception已验证的上下文，开展架构设计、代码实现和测试。简言之，“先要把做什么说明白”属于Inception，“开始按任务实现”才进入Construction。

将AI-DLC与Spec Kit的流程对齐，对应关系如下：

- **constitution**：放在 **Inception最开始**，相当于项目原则和护栏，用于明确团队约束、工程原则、质量标准，与AI-DLC的“规则层”呼应。Spec Kit官方Quick Start也明确，需先创建 `/speckit.constitution`。
- **specify + clarify + checklist**：放在 **Inception核心阶段**，用于定义需求、澄清歧义、验证规格完整性，对应AI-DLC Inception阶段“问清需求、生成结构化规格”的核心目标。Spec Kit官方明确，`/speckit.specify` 聚焦于“做什么（what）”和“为什么做（why）”，不涉及具体技术栈。
- **plan + tasks**：放在 **Inception后半段**，此时需求已稳定，需将其转化为技术实现方案和任务拆分，为进入Construction阶段做准备。Spec Kit的流程顺序也符合这一逻辑——先 `/speckit.plan`，再 `/speckit.tasks`，最后才是 `/speckit.implement`。
- **implement**：放在 **Construction阶段**，此时已完成需求澄清和计划确认，进入具体代码实现环节，对应AI-DLC Construction阶段“怎么实现（how）”的核心目标。

结合Habit Board项目，Spec Kit的落地建议如下：

1. **以AI-DLC为总流程框架**：遵循“先澄清、再计划、再实现、再验证”的节奏，AWS开源仓库的核心逻辑也是将AI-DLC作为三阶段总流程。
2. **将Spec Kit作为Inception阶段的“规格工作台”**：需求启动后，先不直接让AI写代码，而是在Inception阶段内运行 `constitution → specify → clarify → checklist → plan → tasks` 流程，生成完整的规格文档。
3. **将Spec Kit产物提交到仓库**：将Spec Kit生成的spec、plan、tasks等文档，作为Construction阶段的输入，契合AI-DLC“阶段产物持久化、支持跨阶段延续”的要求。

完整流程链路可总结为：

业务意图 → AI-DLC Inception阶段 → 在其中运行Spec Kit
(constitution/specify/clarify/checklist/plan/tasks) → 人工确认规格 → AI-DLC
Construction阶段 → implement/编码/测试 → AI-DLC Operations阶段 → 交付

核心要点：Spec是AI-DLC Inception阶段的核心载体，用于明确“做什么”；Spec Kit则是Inception阶段的“规格工具”，其流程需嵌入Inception阶段，为后续开发提供清晰、可验证的输入。

1.1 Spec Kit 使用报告

参考资料：

- [这才是AI编程的正确姿势：Spec Kit 实战开发演示！7条命令跑通规格驱动开发+强制TDD，从需求到代码全自动！支持 Claude Code、Cursor 哔哩哔哩 bilibili](#)
- [\(29 封私信 / 80 条消息\) 从零理解 GitHub Spec Kit：开发者必看的入门指南 - 知乎](#)
- [Spec Kit - AI-Powered Specification-Driven Development Toolkit](#)

1.1.1 Spec Kit 是什么

<https://github.com/github/spec-kit>

Spec Kit 是 GitHub 在 2025 年推出的开源工具包，用于帮助开发者借助自选 AI 工具落地 Spec-Driven Development（规范驱动开发）。GitHub 官方博客将其描述为：开发者可通过自己偏好的 AI 工具，围绕 spec 组织开发；官方文档则将其实现为一套 `specify` CLI + 一组标准命令。

它并非 IDE、模型，也不是某一agent的专属功能，更像是一个“规范优先”的流程脚手架：先建立项目原则，再撰写规格，接着澄清歧义，然后制定技术方案，拆分任务，最后执行实现。Spec Kit 官方 Quick Start 的标准流程顺序如下：

- `/speckit.constitution`
- `/speckit.specify`
- `/speckit.clarify`
- `/speckit.plan`
- `/speckit.tasks`
- `/speckit.implement`

官方还提供可选命令 `/speckit.analyze` 和 `/speckit.checklist`，用于补充一致性分析和质量检查。

1.1.2 Spec Kit 解决的核心问题

Spec Kit 本质上解决的核心问题是：避免AI一上来直接写代码，先将需求、原则、技术边界和任务拆分固定下来。官方文档明确要求，`/speckit.specify` 阶段需重点描述 `what`（做什么）和 `why`（为什么做），而非技术栈；技术栈、架构选择等内容，需留到 `/speckit.plan` 阶段再明确。

这一设计天然将“需求定义”与“技术设计”拆分开来。对AI开发而言，这至关重要——很多返工并非源于代码实现本身，而是因为需求未明确，就过早进入实现阶段。Spec Kit 的流程正是通过强制拆分这两个阶段，减少此类返工。

1.1.3 Spec Kit 的安装与初始化方式

Spec Kit 官方推荐通过 `specify` CLI 初始化项目，Quick Start 给出的启动命令如下：

```
1 uvx --from git+https://github.com/github/spec-kit.git specify init
   <PROJECT_NAME>
```

或在当前目录初始化：

```
1 uvx --from git+https://github.com/github/spec-kit.git specify init .
```

官方仓库README说明，初始化时可直接指定适配的AI agent，例如 `--ai claude`、`--ai copilot`、`--ai kiro-cli`，以及 `--ai codex --ai-skills`。此外，Spec Kit 支持通用模式，可将模板对接官方列表外的agent。

这表明 Spec Kit 的定位从一开始就是 **agent-agnostic**（与agent无关）：它不是某一AI工具的附属插件，而是一套可适配多个agent的SDD流程层。官方README中的支持列表已覆盖Claude Code、Codex CLI、GitHub Copilot、Kiro CLI、Cursor、Gemini CLI、Generic等多种agent。

1.1.4 Spec Kit 在 Codex 中怎么工作

若使用 **Codex CLI**，官方提供了两条关键信息：

第一，初始化命令为：

```
1 specify init . --ai codex --ai-skills
2 # 完整指令
3 uv tool install specify-cli --from git+https://github.com/github/spec-
  kit.git@v0.6.0
4 specify init --here --ai codex --ai-skills
```

第二，Spec Kit 在Codex中并非传统slash command形式，而是以 **agent skills** 方式安装，调用形式变为：

- `$speckit-constitution`
- `$speckit-specify`
- `$speckit-plan`
- `$speckit-tasks`
- `$speckit-implement`

这是Spec Kit README明确写出的Codex适配方式；同时OpenAI的Codex文档说明，Codex的技能通过 **SKILL.md** 打包指令、资源和可选脚本，并采用渐进式披露（progressive disclosure）机制，仅在命中相关任务时加载完整技能内容。（github.com ； developers.openai.com ）

因此，Spec Kit + Codex 的关系并非“Codex内置了Spec Kit”，而是 Spec Kit 将一套SDD流程封装为Codex的技能包。

1.1.5 Spec Kit 会生成什么

Spec Kit README清晰描述了生成过程：

执行 `constitution` 后，会创建或更新：

```
1 .specify/memory/constitution.md
```

执行 `specify` 后，会为当前功能新建一个编号目录，例如：

```
1 .specify/specs/001-create-taskify/spec.md
```

README还展示了完整的生成结构，包括：

- `.specify/memory/constitution.md`
- `.specify/specs/<feature>/spec.md`
- `plan.md`
- `research.md`
- `quickstart.md`
- `contracts/`
- `tasks.md`

也就是说，Spec Kit的输出并非仅一份PRD，而是一整套 **从原则到规格、计划再到任务** 的结构化工件。

1.1.6 Spec Kit 的核心流程应该怎么理解

用最简单的话概括，Spec Kit的标准节奏是：

先立规矩，再写规格，再补清楚，再定技术方案，再拆任务，最后实现。 这并非改写总结，而是GitHub官方文档和README的实际流程映射。

其中几个关键点需重点关注：

- `/speckit.clarify` 官方建议放在 `/speckit.plan` 之前，README明确说明此举是为了减少下游返工。
- `/speckit.tasks` 会生成按用户故事组织的任务，包含依赖顺序、并行标记 `[P]`、具体文件路径、测试优先顺序和阶段检查点。
- `/speckit.implement` 会先验证`constitution/spec/plan/tasks`等前置条件是否齐全，再按任务顺序执行实现。

因此，Spec Kit 并非“帮你写一份spec”，而是**把spec变成后续实现的入口工件**。

1.2 把 Vue3 示例改写成 Spec Kit 流程

以下将轻量Vue3项目，直接改造成 **Spec Kit + Codex** 的标准落地方式。

项目仍沿用此前示例：

Habit Board（习惯打卡板）

目标功能不变：

- 新增习惯
- 每日打卡
- 查看连续打卡天数
- `localStorage` 本地持久化

为保持示例简洁，仍使用 **Vue 3 + Vite**。Vue官方Quick Start说明，`npm create vue@latest` 会调用 `create-vue`，并可按需勾选Router、Pinia、Vitest、ESLint、Prettier等功能；Vite官方文档说明，当前Vite要求Node.js版本为 **20.19+** 或 **22.12+**。

1.2.1 第一步：搭建 Vue3 项目

```
1 npm create vue@latest habit-board
2 cd habit-board
3 npm install
4 npm run dev
```

若需最轻量版本，初始化时可勾选：

- Vue Router：否
- Pinia：否
- Vitest：是
- ESLint：是
- Prettier：可选

本示例为本地单页应用，无需提前增加复杂度。Vue官方脚手架支持创建时灵活选择这些选项。

1.2.2 第二步：初始化 Spec Kit 到仓库

若配合Codex使用，最直接的初始化方式为：

```
1 uvx --from git+https://github.com/github/spec-kit.git specify init . --ai
  codex --ai-skills
```

此步骤完成后，Spec Kit会将适配Codex的技能和流程模板装入项目；官方README说明，Codex模式下会通过skills方式调用Spec Kit，而非普通slash commands。

1.2.3 第三步：编写 constitution，明确项目原则

进入Codex后，不要立即生成Vue组件，先执行第一条命令：

```
1 $speckit-constitution
2 为这个项目建立原则：
3 - Vue 3 + Vite，优先简单实现
4 - v1 不引入后端
5 - 状态尽量局部化，先不用 Pinia
6 - UI 保持最小依赖
7 - 关键逻辑必须有 Vitest 单测
8 - 所有实现以可读性和可维护性优先
```

此步骤用于定义项目的长期原则和治理规则，即“所有功能都需遵守的总规则”。

根据Spec Kit官方文档，此步骤的结果会写入 `.specify/memory/constitution.md`，并作为后续specification、planning、implementation的基础约束。

constitution中适合写入：

- 产品原则
- 体验原则
- 工程原则
- 测试与质量门槛
- 性能要求
- 不允许做的事

此步骤对应AI-DLC的 Inception阶段最开始的“先立原则”。

1.2.4 第四步：用 **specify** 明确“做什么”，不涉及“怎么做”

下一步执行：

```
1 $speckit-specify
2 构建一个 Habit Board Web 应用，帮助用户维护日常习惯。
3 用户可以新增习惯、每天打卡、查看连续打卡天数，并且刷新页面后数据仍然保留。
4 这个版本只面向单用户、本地浏览器使用，不需要账号系统，不需要云同步。
5 重点是清晰、稳定、易维护，而不是功能全面。
```

Spec Kit官方明确要求：**specify** 阶段聚焦 **what**（做什么）和 **why**（为什么做），不提前灌入技术栈细节。

specify阶段适合写入：

- 用户是谁
- 要解决什么问题
- 核心场景
- 需要哪些功能
- 验收标准

不适合写入：

- Next.js、Supabase、Tailwind等技术细节

此步骤结束后，按默认模板，仓库中通常会生成一个编号功能目录，例如：

```
1 .specify/specs/001-habit-board/spec.md
```

README中的Taskify示例即采用这种编号分支+编号spec目录的模式。

1.2.5 第五步：用 **clarify** 澄清模糊点

不要急于执行plan，先让Codex运行：

```
1 $speckit-clarify
2 重点澄清以下问题：
3 - 连续打卡天数的计算规则
4 - 是否允许一天多次打卡
5 - 删除习惯后历史是否保留
6 - 习惯名称是否允许重复
7 - 空状态、首次使用状态如何展示
```

Spec Kit README明确建议：**clarify** 需在 **plan** 前执行，通过结构化方式减少后续返工，**补充和消除specification中的歧义**。

此步骤非常接近AI-DLC的 **Requirements Analysis（需求分析）**：不直接做假设，而是补齐需求中的空白处。

可以不输入prompt， AI自动判断模糊点

1.2.6 第六步：用 plan 引入 Vue3 / Vite 技术方案

需求明确后，执行：

```
1 $speckit-plan
2 技术方案如下：
3 - 前端框架：Vue 3
4 - 构建工具：Vite
5 - 使用 Composition API 和 <script setup>
6 - 不引入 UI 组件库
7 - 持久化：localStorage
8 - 测试：Vitest
9 - 首版不使用 Vue Router 和 Pinia
10 请生成最小但完整的实现计划，并避免过度设计。
```

Spec Kit官方文档明确将此步骤定义为“Create a Technical Implementation Plan（制定技术实现计划）”，即在此阶段提供技术栈和架构选择。

README说明，plan阶段通常会生成：

- **plan.md**
- **research.md**
- **quickstart.md**
- 可能的 **contracts/**
- 更新后的功能目录内容

也就是说，此步骤并非“简单撰写技术说明”，而是正式生成 **技术实现工件**。

plan阶段适合写入：

- 前端框架
- 状态管理
- 数据持久化

- API方案
- 目录结构
- 组件边界
- 测试策略

`/speckit.plan` 的核心职责，是将 `spec.md` 中的“要做什么”翻译为“准备怎么实现”，并同步产出一组配套文档。Spec Kit官方对该阶段的描述是：读取功能spec，检查是否符合constitution，将需求转化为技术方案，并生成数据模型、API契约和关键验证场景等支持性文档。`/speckit.tasks` 后续会基于这些文件拆分任务。

plan阶段生成的5类核心文件说明

1. `/plan.md`：主技术方案文档，用于明确“MVP准备怎么做”，通常包含技术栈、架构选择、页面/模块划分、实现边界、阶段性策略及选择依据。对本项目而言，它相当于“AI学习计划助手MVP的总施工图”，开发时可作为整体方向指引，评审时可判断方案是否过度工程化，也是生成 `tasks.md` 的核心输入。
2. `/research.md`：技术调研与关键决策说明，用于明确“为什么这么选”，解决不确定项、比较方案、记录取舍依据。对本项目而言，会回答“为什么用Next.js而非纯Vite”“为什么先用mock+localStorage”等问题，是后续解释技术选型的核心参考。
3. `/data-model.md`：数据模型文档，用于明确“系统核心对象的结构与关联”，定义 `UserGoal`、`StudyPlan` 等实体的字段、约束、状态枚举及关系。前端编写TypeScript类型时可直接参考，后续对接后端或数据库时可作为数据结构基础，`tasks.md` 也会基于这些实体拆分实现任务。
4. `/quickstart.md`：快速验证与上手文档，聚焦“关键场景如何快速跑通、如何验证功能有效性”，包含项目启动、核心场景操作、功能验证路径等内容。既是开发者的“最快自测脚本”，也是评审者的“最短验收路径”，为后续implement阶段提供关键流程指引。
5. `/openapi.yaml`：接口契约文档，用于明确“前后端通信规则及输入输出结构”，即使当前仅做前端MVP，也能提前固定“AI计划生成”“任务状态更新”等接口的请求/响应结构，为前端mock编写、后端后续实现及联调提供统一依据。

总结：5类文件可简化为“总方案（plan.md）、决策原因（research.md）、数据结构（data-model.md）、关键验证路径（quickstart.md）、接口约定（contracts/openapi.yaml）”。

1.2.7 第七步：用 tasks 拆分可执行小任务

完成plan后，执行：

```
1 $speckit-tasks
```

根据Spec Kit官方说明，此步骤会生成 `tasks.md`，任务包含以下要素：

- 按用户故事分阶段组织
- 依赖顺序
- 可并行标记 [P]
- 具体文件路径
- 测试优先顺序
- 每个阶段的checkpoint（检查点）

对Habit Board项目，合理的任务拆分如下：

- T1: 定义Habit数据结构与storage模块
- T2: 实现HabitForm组件
- T3: 实现HabitList组件
- T4: 实现打卡逻辑与连续打卡天数（streak）计算
- T5: 处理重复名称与空状态
- T6: 为核心逻辑补充Vitest单测
- T7: 构建与交付检查

上述任务名为基于项目需求的合理重组，而“tasks.md需包含依赖、路径、测试顺序、检查点”是官方文档明确要求的。

1.2.8 第八步：用 analyze / checklist 审核前置条件

若需提升流程稳定性，建议在implement前执行：

```
1 $speckit-analyze
2 $speckit-checklist
```

官方README将 **analyze** 定义为 **Cross-artifact consistency & coverage analysis**（跨工件一致性与覆盖度分析），**checklist** 用于检查需求完整性、清晰度和一致性。

对本Vue3项目而言，这两步可重点发现三类问题：

- spec中已定义，但tasks中遗漏的功能
- plan中技术复杂度过高的设计
- 测试范围未覆盖streak、persistence等核心逻辑的情况

1.2.9 第九步：用 implement 正式进入编码

前置步骤确认无误后，执行：

```
1 $speckit-implement
```

Spec Kit官方说明，**implement** 会先验证constitution/spec/plan/tasks是否齐全，再按任务顺序执行。同时提醒：agent会实际运行本地CLI（如**npm**），需确保本地依赖已准备就绪。

对Habit Board项目，不建议一次性“全部实现”，推荐遵循Quick Start建议：**复杂项目分阶段实现，先完成核心功能再叠加细节**（此建议为GitHub官方文档明确写出）。

更稳妥的分阶段实现方式：

第一轮实现：

- 数据结构定义
- localStorage封装
- HabitForm组件
- HabitList组件

第二轮实现：

- 打卡逻辑
- streak计算
- 边界条件处理
- 测试与构建

1.2.10 第十步：将 Spec Kit 流程映射回 AI-DLC

将Spec Kit流程与AI-DLC三阶段对应，如下：

Inception（启动阶段）

- constitution（立原则）
- specify（定需求）
- clarify（澄清歧义）
- checklist（验证需求）
- plan（定技术方案）
- tasks（拆任务）

Construction（构建阶段）

- analyze（质量闸门，验证方案）
- implement（编码实现）
- 本地测试/构建修复

Operations（运营阶段，轻量前端项目最小版本）

- `npm run build`（生产构建）
- 生成简要部署说明
- 发布检查清单

AWS对AI-DLC的定义为：Inception负责规划与架构，Construction聚焦设计与实现，Operations覆盖部署与监控；这与Spec Kit“先规范、澄清、计划、任务，再实现”的流程可顺利对齐。

1.3 最终结论

核心总结：

Spec Kit 是一套agent无关的SDD流程脚手架；它最适合放在AI agent之上，负责把“立原则 → 写规格 → 澄清 → 计划 → 任务 → 实现”变成标准路径。

对Vue3轻量项目而言，最自然的落地方式为：

Vue3/Vite搭建项目 → Spec Kit初始化为Codex模式 → constitution → specify → clarify → plan → tasks → analyze/checklist → implement → build/release。

该流程既符合Spec Kit官方工作流，也能与AI-DLC的Inception/Construction/Operations三阶段顺利对齐。

1.4 推荐使用顺序

实际落地时，可直接按以下顺序执行Spec Kit相关命令，确保流程规范、落地高效：

```
1 /speckit.constitution
2 /speckit.specify
3 /speckit.clarify
4 /speckit.plan
5 /speckit.tasks
6 /speckit.analyze
7 /speckit.implement
```

说明：大多数AI助手可直接使用 `/speckit.*` 格式命令；若使用Codex CLI的skills模式，官方规范格式为 `$speckit-*`。

1.5 整套 Prompt 模板

1.5.1 `/speckit.constitution`

此阶段核心是定义项目宪法，而非具体功能。官方建议在此明确代码质量、测试标准、体验一致性等核心原则，后续所有spec、plan、implement操作均需遵循这些原则。推荐直接使用以下模板：

```
1 /speckit.constitution
2 本项目是一个 Vue3 后台页面生成助手，必须遵循以下原则：
3
4 1. 简单起步原则
5 - MVP 阶段只允许单 Agent 架构。
6 - 不允许在第一阶段引入多 Agent 协作网络。
7 - 如果能力不足，优先新增 Skill、模板、检查清单或固定工作流，而不是新增 Agent。
8
9 2. 工程简洁原则
10 - 项目总拆分控制在最小必要范围内，优先保持 ≤3 个主要项目或模块边界。
11 - 不做未来预埋，不为了“可能以后会用”提前引入复杂抽象。
12 - 优先直接使用 Vue3、Vite、Vue Router、Pinia、Element Plus 官方能力。
13
14 3. 规范一致性原则
15 - 所有生成页面必须遵守统一的后台页面结构规范：
16   搜索区、表格区、分页区、详情抽屉、编辑弹窗、空态、错误态、加载态。
17 - 组件命名、路由命名、接口命名、类型命名必须一致。
18
19 4. 测试与验证原则
```

- 所有生成结果至少通过 `lint`、`type-check` 和最小 `smoke test`。
 - 先有验收标准，再开始实现。
 - 输出必须包含人工审核点，不允许直接自动提交生产代码。
- ### 5. 可观测性原则
- 记录需求输入、页面 `schema`、模板选择依据、工具调用记录、生成结果摘要、错误信息。
 - 能够定位失败发生在需求理解、模板选择、接口映射还是代码生成阶段。
- ### 6. 安全与稳定性原则
- 所有表单输入要有基本校验。
 - 所有异步请求要有 `loading`、`error`、`empty` 三态。
 - 不生成不透明的“魔法层”代码，优先可读、可改、可维护。
- ### 7. 演进原则
- 只有当单 `Agent` 在上下文长度、并行效率、角色隔离方面被证明确实不足时，才允许升级到多 `Agent`。
 - 升级顺序必须为：单 `Agent` → 单 `Agent` + `Skill` → 单 `Agent` + `Workflow` → 主 `Agent` + `Subagents` → 多 `Agent`。

1.5.2 /speckit.specify

官方建议此阶段仅聚焦what（做什么）/ why（为什么做），不提前明确技术栈。推荐直接使用以下模板：

- ```
1 /speckit.specify
2 构建一个面向前端团队内部使用的 AI 页面生成助手。
3
4 目标：
5 用户输入一段后台需求描述后，系统能够自动生成一个 Vue3 管理后台页面的初始实现草稿，帮助减少重复搭建页面的工作量。
6
7 第一阶段要解决的问题：
8 - 后台列表页、详情页、编辑弹窗页面经常重复开发
9 - 页面结构相似，但字段、接口、交互细节不同
10 - 需求文档到页面骨架之间存在大量机械性劳动
11 - 团队希望先提升页面脚手架生成效率，而不是一次性全自动完成整个项目
12
13 核心用户：
14 - 前端工程师
15 - 低年级开发同学
16 - 需要快速搭建后台页面原型的业务开发者
17
18 第一阶段范围：
19 - 支持列表页、详情抽屉、编辑弹窗三种常见后台模式
20 - 支持搜索、筛选、分页、表格列展示、表单字段生成
21 - 输出 Vue3 页面草稿、组件草稿、API 草稿、类型定义草稿、验收清单
22 - 生成结果必须可预览、可人工调整、可审核后导出
```

```
23
24 非目标：
25 - 第一阶段不做设计稿自动解析
26 - 第一阶段不做全项目级代码生成
27 - 第一阶段不直接自动提交代码到业务仓库
28 - 第一阶段不引入多 Agent 协作网络
29
30 成功标准：
31 - 输入典型后台页面需求后，能生成结构正确、可运行的页面草稿
32 - 生成功能覆盖主流后台页面骨架
33 - 人工修改量明显低于从零开发
34 - 页面基础结构、命名和交互状态保持一致
```

### 1.5.3 /speckit.clarify

该命令的核心作用是澄清需求歧义，可传入重点关注的澄清领域。建议分两轮执行，分别聚焦业务边界和工程实现，确保需求无模糊点。

#### 第一轮：业务边界澄清

```
1 /speckit.clarify
2 请重点澄清以下业务边界与验收问题：
3
4 1. 第一阶段支持哪些页面类型，明确不支持哪些页面类型？
5 2. 需求输入是自然语言描述、结构化字段配置，还是二者兼容？
6 3. 页面草稿生成的最小可交付单位是什么：单页面、单模块，还是页面 + API + 类型定义？
7 4. 表单字段规则是否需要覆盖必填、长度、枚举、日期、数字范围等常见校验？
8 5. 列表页是否必须包含搜索、筛选、分页、批量操作、详情查看这些能力中的全部，还是按需生成？
9 6. 详情展示与编辑弹窗是否必须共存？
10 7. 用户对生成结果的后续操作是什么：预览、编辑、复制、导出、落盘？
11 8. 验收指标如何定义：可运行率、生成成功率、人工修改率、页面一致性、生成耗时？
12 9. 第一阶段是否需要权限显隐、按钮级权限控制？
13 10. 页面是否必须兼容移动端，还是只面向桌面后台系统？
```

## 第二轮：工程与数据澄清

```
1 /speckit.clarify
2 请重点澄清以下工程实现问题：
3
4 1. API 规范是否默认为 REST 风格？
5 2. 分页参数和返回结构是否需要统一约定？
6 3. 类型定义由系统自动推断，还是允许用户补充字段类型？
7 4. 是否需要 mock 数据输出？
8 5. 是否需要生成 Pinia store 与 composables？
9 6. 是否需要自动生成路由配置与菜单元信息？
10 7. 页面是否必须具备 loading、empty、error 三态？
11 8. 是否要求输出测试骨架？
12 9. 生成代码是否以项目内文件树形式输出？
13 10. 第一阶段是否只生成草稿文件，不直接写入真实业务仓库？
```

### 1.5.4 /speckit.plan

此阶段核心是制定技术实现方案，官方明确要求在此步骤确定具体技术细节。提供两版模板，优先使用单Agent MVP版，仅在单Agent无法满足需求时考虑升级。

#### 版本 A：单 Agent MVP 版（优先使用）

```
1 /speckit.plan
2 请为该项目生成一个以“单 Agent MVP”为核心的技术实现方案，要求如下：
3
4 前端技术栈：
5 - Vue 3
6 - Vite
7 - TypeScript
8 - Vue Router
9 - Pinia
10 - Element Plus
11
12 后端/Agent 服务：
13 - Node.js
14 - 提供一个简单 API 服务，负责接收需求、调用模型、执行工具、输出草稿结果
15
16 运行时架构要求：
17 - 第一阶段必须采用单 Agent
18 - 单 Agent 负责：需求解析、页面 schema 提取、模板选择、代码草稿生成、结果自检、输出审核材料
19 - 不允许多 Agent 协作网络
20 - 如果能力不足，优先通过 Skill、模板库、规则清单和固定 workflow 增强
21
22 建议能力模块：
23 - vue3-admin-patterns skill：后台列表页/详情页/弹窗规范
24 - api-contract-checklist skill：接口命名、分页、字段、错误处理规范
```

```
25 - ui-review-checklist skill: loading、empty、error、校验、交互反馈检查清单
26
27 工具设计要求：
28 - read_component_library
29 - load_page_template
30 - generate_vue_files
31 - generate_api_contracts
32 - run_lint_and_typecheck
33 - run_smoke_test
34 - save_draft_files
35 - request_human_review
36
37 输出物要求：
38 - 页面 schema
39 - Vue 页面草稿
40 - 组件草稿
41 - API 层草稿
42 - 类型定义草稿
43 - mock 数据草稿
44 - 验收清单
45 - 风险提示
46
47 工程约束：
48 - 保持最小项目数量，避免过度拆分
49 - 不做未来预埋
50 - 不额外封装框架壳层
51 - 先有 contracts / tests / 验收标准，再进入实现
52 - 必须考虑可观测性：记录 prompt、模板选择、工具调用、错误归因、耗时与成本
53
54 请在 plan 中明确：
55 1. 系统上下游边界
56 2. 前后端目录结构
57 3. Agent 主循环
58 4. Skill 的职责边界
59 5. 工具调用链路
60 6. 页面生成数据流
61 7. 失败回退策略
62 8. 人工审核点
63 9. 后续升级到 Subagent / 多 Agent 的触发条件
```

## 版本 B：多 Agent 升级版（单Agent不足时使用）

```
1 /speckit.plan
2 请基于现有单 Agent MVP，生成一个“第二阶段升级方案”，目标是评估是否有必要升级
 为多 Agent。要求如下：
3
4 现状：
5 - 已有单 Agent 页面生成链路
```

```
6 - 已有页面模板、Skill、工具调用链路
7 - 已有基础可观测性和人工审核流程
8
9 请重点评估以下升级条件：
10 1. 单 Agent 是否存在上下文过载？
11 2. 是否存在天然可并行的子任务？
12 3. 是否需要不同角色视角拆分？
13 4. 是否有明确指标证明多 Agent 的收益大于协调成本？
14
15 如果升级，请优先输出：
16 - 主 Agent + Subagents 方案
17 而不是直接输出完全自由协作的多 Agent 网络
18
19 候选子角色：
20 - spec-analyst: 负责需求结构化与页面 schema
21 - ui-generator: 负责 Vue 页面和组件代码草稿
22 - api-mapper: 负责接口层与类型映射
23 - qa-reviewer: 负责验收检查与测试建议
24
25 请明确：
26 - 每个子 Agent 的输入输出
27 - 各自工具权限
28 - 各自 Skill 依赖
29 - 聚合与回收机制
30 - 任务冲突与文件冲突如何处理
31 - 何时不应该升级为多 Agent
32 - 升级后的成本、调试复杂度和收益评估方法
```

### 1.5.5 /speckit.tasks

官方定义此命令用于将技术方案拆解为可执行任务，核心是确保任务具体、可落地。推荐使用以下模板，避免任务拆分过粗或过虚：

```
1 /speckit.tasks
2 请基于当前 spec 和 plan，输出一个可执行的任务清单，要求如下：
3
4 1. 任务要按阶段组织：
5 - Phase 1: 项目初始化
6 - Phase 2: 需求输入与结果预览
7 - Phase 3: 单 Agent 主链路
8 - Phase 4: Skill 与模板库接入
9 - Phase 5: 代码草稿生成
10 - Phase 6: 校验与可观测性
11 - Phase 7: 人工审核与导出
12 - Phase 8: 评估与迭代
13
14 2. 每个任务必须包含：
15 - 任务编号
```

```
16 - 任务标题
17 - 目标说明
18 - 涉及文件/目录
19 - 前置依赖
20 - 完成标准
21
22 3. 优先拆成小步任务，不要把“大而全”的任务放在一起
23
24 4. 对可以并行的任务标记 [P]
25
26 5. 必须体现 test-first / contract-first 思路：
27 - 先 schema / contracts
28 - 再 smoke test / type-check
29 - 再生成实现
30
31 6. 必须体现简单起步原则：
32 - 第一阶段只做单 Agent
33 - Skill 优先于多 Agent
34 - 多 Agent 相关任务只能以“未来升级条件”形式保留，不能进入 MVP 主任务流
35
36 7. 请在任务清单最后补充：
37 - MVP 完成定义
38 - 评估指标采集任务
39 - 失败样本归因任务
```

### 1.5.6 /speckit.analyze

该命令为可选的方案校验步骤，核心是检查方案是否存在过度工程化等问题，确保方案简洁、可落地。推荐使用以下模板，重点聚焦MVP阶段的合理性：

```
1 /speckit.analyze
2 请重点分析当前 spec、plan、tasks 是否存在以下问题：
3
4 1. 是否过度工程化？
5 2. 是否在 MVP 阶段引入了不必要的多 Agent 复杂度？
6 3. 是否存在 future-proofing 和过早抽象？
7 4. 是否违反“先 Skill、后 Agent”的原则？
8 5. 是否存在任务拆分过粗、不可执行的问题？
9 6. 是否缺少 contracts、测试、验收标准或人工审核点？
10 7. 是否缺少必要的可观测性设计？
11 8. 是否存在 Vue3 工程层面的不必要封装？
12
13 请按“问题 - 风险 - 建议修改”的格式输出。
```

### 1.5.7 /speckit.implement

此阶段为方案落地执行，核心是严格按照任务清单生成可运行的代码草稿，避免失控。推荐使用以下模板，添加约束确保生成结果符合预期：

```
1 /speckit.implement
2 请严格按照 tasks.md 执行实现，要求如下：
3
4 1. 按任务顺序逐步实现，不要跳过阶段
5 2. 一次只处理一个或一组强相关的小任务
6 3. 每完成一组任务后，先总结变更，再继续下一组
7 4. 先创建或完善 contracts、schema、tests，再实现源码
8 5. 严格遵守 constitution 中的简单起步原则
9 6. MVP 阶段禁止擅自引入多 Agent 结构
10 7. 如果发现单 Agent 能力不足，先提出：
11 - 应新增什么 Skill
12 - 应新增什么模板
13 - 应新增什么检查清单
14 而不是直接改成多 Agent
15 8. 所有生成结果必须保持 Vue3 + TypeScript + Vite 工程一致性
16 9. 所有页面都要考虑 loading、empty、error 三态
17 10. 输出每一步的修改摘要、风险点和待人工确认项
```

## 1.6 迭代时的补充模板

Spec Kit官方主流程主要覆盖新特性从constitution到implement的完整链路，实际项目中常存在需求变更、方案调整、范围收缩等场景。官方社区实践表明，可继续使用plan、tasks等命令更新现有内容，

其中 **/speckit.constitution** 作为项目原则层，需谨慎修改、尽量少改，其余命令可在每轮迭代中灵活调整。

补充3个高频迭代场景的模板，覆盖需求变更、方案收缩、多Agent升级评估：

### 1.6.1 需求变更模板

```
1 /speckit.specify
2 请在当前功能范围内追加以下需求，并保持 MVP 边界不失控：
3
4 新增需求：
5 - 支持列表页的批量操作栏
6 - 支持表格列自定义显隐
7 - 支持详情页中的操作日志区域
8
9 要求：
10 - 不要扩大到设计稿自动解析
11 - 不要引入多 Agent
12 - 明确哪些是本次新增范围，哪些仍然是非目标
13 - 更新成功标准与验收边界
```

### 1.6.2 方案收缩模板

```
1 /speckit.plan
2 请收缩当前实现方案，目标是降低复杂度并保证 MVP 可落地：
3
4 要求：
5 - 删除非必要抽象层
6 - 删除未来预埋设计
7 - 合并职责重复的模块
8 - 保持单 Agent
9 - 用 Skill 和模板替代复杂编排
10 - 明确哪些内容延期到第二阶段
```

### 1.6.3 从单 Agent 升级评估模板

```
1 /speckit.analyze
2 请评估当前项目是否已经满足升级到主 Agent + Subagents 的条件。
3
4 请重点判断：
5 1. 单 Agent 是否已经成为瓶颈？
6 2. 哪些子任务天然适合隔离上下文？
7 3. 是否存在可观测的收益，例如更高成功率、更低修改率、更短耗时？
8 4. 多 Agent 带来的协调成本是否可接受？
9
10 请输出：
11 - 结论：暂不升级 / 可以升级
12 - 证据
13 - 推荐架构
14 - 不推荐直接升级的原因
```

## 1.7 最实用的使用建议

---

结合官方Quick Start建议和实际落地经验，最推荐的执行流程如下：

首先执行以下命令，完成需求澄清、方案制定和任务拆解，并校验方案合理性：

```
1 /speckit.constitution
2 /speckit.specify
3 /speckit.clarify
4 /speckit.plan
5 /speckit.tasks
6 /speckit.analyze
```

确认技术方案无过度工程化、符合MVP原则后，再执行实现命令：

```
1 /speckit.implement
```

核心提示：官方明确建议，复杂项目需分阶段实现，优先完成核心能力搭建，再逐步新增功能，避免一次性开发导致的失控和冗余。

---

# 第五篇 Figma MCP

## 1. Figma MCP 是什么

Figma 官方把 MCP Server 定位成“把 Figma 直接带入开发工作流”的桥梁：AI agent 可以读取 Figma 文件中的 `components`、`variables`、`layout data` 等结构化信息，用这些信息生成更贴近设计系统的代码；反过来，在 `remote` 模式下，agent 还可以把内容直接写回 Figma 画布。

Figma 也明确说明：**MCP 本身不是一键把设计变成完美代码的工具**，它负责把结构化设计上下文送给 AI，最终代码仍由 AI 客户端结合你的 `prompt` 和代码库来生成。

核心总结：MCP 的核心作用是传递结构化设计上下文，而非直接生成完美代码，最终代码由 AI 客户端结合提示词和代码库生成。

## 2. remote MCP 和 desktop MCP 的区别

Figma 目前有两种 MCP 形态：`remote MCP server` 和 `desktop MCP server`。官方推荐大多数用户优先使用 `remote`，因为它连接的是 Figma 托管的远程端点，功能最全，而且不要求安装 Figma Desktop App。

两者核心区别如下：

- **remote MCP**：是 **链接驱动**。复制整个文件链接，或者某个 `frame / layer` 的链接，贴给 agent，agent 再基于这个 URL 获取上下文。官方明确写了 `remote` 是 **link-based**。
- **desktop MCP**：更偏 **当前选中内容驱动**。主要理解你在 Figma Desktop 里当前选中的对象，也可以配合链接，但核心是 **selection-based**。

适用场景区分：

- 适合选 `remote` 的场景：主要在浏览器里用 Figma、想要全部可用工具、习惯把 `frame/layer` 链接贴给 Claude Code、Cursor、Codex、VS Code 等 `agentic tool`。
- 适合 `desktop` 的场景：长期在 Figma Desktop 里工作，希望 agent 直接获取“当前选中节点”的上下文，或有特定组织/企业场景必须走本地。

## 3. 接入方式、认证和可用范围

### 3.1 remote MCP

官方远程端点是 <https://mcp.figma.com/mcp>（注：该链接当前提示“link dead”，无法正常访问）。`remote` 方案一般通过编辑器或 AI 客户端里的插件 / MCP 配置接入，然后走 Figma 账号授权流程。Figma 官方示例里已包含 Claude Code、Codex、Cursor、VS Code、Gemini CLI 等接入方式。

## 3.2 desktop MCP

desktop 方案需要安装 **Figma Desktop App**，并在 Dev Mode 中开启 MCP server。官方给出的本地地址是 <http://127.0.0.1:3845/mcp>（注：该链接当前提示“invalid link”，无法正常访问）。

## 3.3 座席与权限

官方文档当前给出的规则是：

- **remote server**：所有 seat / plan 都可用，但额度和能力不同。
- **desktop server**：需要付费计划中的 **Dev 或 Full seat**。
- **写回 Figma (use\_figma 写操作)**：通常需要 **Full seat**；Dev seat 更适合只读 workflow，例如读取 design context、variables、screenshots、metadata 等。

另外，Figma 当前说明 MCP 会逐步变成 **usage-based paid feature**，但文档仍写着 **beta 期间免费**。

# 4. 各类工具的详细解释

以下按功能分组，详细说明 Figma MCP 各类工具的作用、用途及核心细节。

## 4.1 读取设计上下文类

### 4.1.1 `get_design_context`

这是最核心的“**把设计转成结构化上下文**”工具。官方明确说明：它会返回某个 layer 或 selection 的 design context，**默认输出是 React + Tailwind 表示**，但可通过 prompt 要求改成 Vue、HTML + CSS、iOS 等其他框架/风格。它支持 **Figma Design** 和 **Figma Make**。

可以把它理解成：“不是最终代码，而是 AI 可理解的设计表达层。”

**典型用途：**

- 让 AI 根据某个 frame 生成 Vue 组件
- 让 AI 根据某个 selection 输出 HTML + CSS
- 结合 Code Connect，让 AI 优先复用你真实代码库中的组件，而非凭空猜测组件结构。

**重要细节：**

- desktop 支持“基于当前选中内容”的 prompting；
- remote 则必须提供 **frame/layer 链接**。

### 4.1.2 `get_variable_defs`

这个工具负责提取选区里用到的 **variables 和 styles**，比如颜色、间距、字体等 token。它是把“设计稿里的视觉值”转换成“AI 可引用的 design token 上下文”的关键入口。

**典型用途：**

- 问 AI：“这个 frame 用了哪些颜色变量和间距变量？”
- 让 AI 在生成代码时尽量引用 token 名，而非写死数值

- 做 design system 对齐，检查设计稿是否真正使用了变量。

#### 4.1.3 get\_metadata

这个工具返回一个 **稀疏 XML**，只包含基础结构信息：比如 layer ID、名称、类型、位置、尺寸等。官方强调它适合 **非常大的设计文件**：先用它拿到轻量级大纲，再让 agent 对局部节点继续调用 `get_design_context`，避免一次把超大设计全塞进上下文。

**适用场景：**

- 超大页面
- 多 frame 文件
- 想先做结构扫描，再做局部精读。

#### 4.1.4 get\_screenshot

这个工具让 agent 对当前 selection 截图。官方建议通常 **保持开启**，因为截图能帮助模型保留布局和视觉保真度；只有在特别担心 token 消耗时，才考虑关闭。它支持 **Figma Design** 和 **FigJam**。

**可以把它理解成：**“结构化上下文 + 视觉截图”的双保险。

因为纯结构化信息对尺寸、对齐、视觉层级的感知有时不如图片直观，所以很多设计转代码场景下，`get_screenshot` 很有价值。

#### 4.1.5 get\_figjam

这是 FigJam 版的 `get_metadata`。它会以 XML 的方式返回 FigJam 图中的节点元数据，而且还包含节点截图。适合把流程图、架构图、头脑风暴板等 FigJam 内容提供给 AI 做开发上下文。

**适用场景：**

- 从 FigJam 里的流程图生成接口流程说明
- 从架构草图生成系统模块文档
- 把产品讨论白板喂给 AI 辅助开发。

## 4.2 Code Connect 相关工具

#### 4.2.1 get\_code\_connect\_map

这个工具读取当前选中实例与代码库组件之间的映射关系。返回内容包括：组件名、代码位置、snippet、版本来源、框架标签等。remote 还可以通过 `clientFrameworks` 和 `clientLanguages` 控制返回哪套映射。

**本质作用：**让 AI 知道“这个 Figma 实例在你项目里其实对应 `Button.tsx` / `Button.vue`，不是要现编一个新的按钮”。

这对真正落地的 design-to-code 非常关键。

#### 4.2.2 add\_code\_connect\_map

这个工具用于新增映射：把某个 Figma node ID 映射到代码库中的具体组件。官方说这样能显著提升 design-to-code 输出质量，因为模型能更准确识别和复用项目里的真实组件。

适用场景：

- 刚完成一个组件的 Figma ↔ code 对应关系
- 想补齐团队设计系统的映射资产
- 想让 AI 后续更稳定复用组件。

#### 4.2.3 get\_code\_connect\_suggestions

这个工具会让 Figma 帮你检测并建议哪些 Figma 组件可以映射到代码组件。官方 Code Connect skill 中，它通常被用作“发现尚未映射组件”的第一步。

适用场景：

- 扫描未映射组件
- 建立或补齐 Code Connect
- 做设计系统治理。

#### 4.2.4 send\_code\_connect\_mappings

这个工具通常在 get\_code\_connect\_suggestions 之后使用，用于确认映射结果。可以理解为“把建议映射正式提交/确认”的后续动作。

#### 4.2.5 get\_context\_for\_code\_connect

这个工具主要出现在官方 Skill: Code Connect 工作流文档里。它的作用是根据 fileKey + nodeId 去读取某个 Figma 组件的 属性定义，包括 TEXT、BOOLEAN、VARIANT、INSTANCE\_SWAP 等，用来帮助生成 .figma.ts 之类的 Code Connect 模板。

它的定位不是普通“设计转代码”主入口，而是更偏 Code Connect 模板生成/维护。

换句话说：

- get\_design\_context 更偏“把设计交给 AI 生成代码”
- get\_context\_for\_code\_connect 更偏“把组件属性结构交给 AI，建立 Figma 组件和代码组件的正式连接层”

### 4.3 写回 Figma / 创建设计类

#### 4.3.1 use\_figma

这是 remote 侧最重要的“通用写画布工具”。官方定义很明确：它可以在 Figma 文件里 创建、编辑、删除或检查 各类对象，包括 pages、frames、components、variants、variables、styles、text、images 等。并且在合适的时候，agent 会先去检查设计系统里是否已有可复用组件，而不是从零乱造。

典型用途：

- 新建一个 screen/frame

- 更新某个组件样式
- 批量建立 token / variable collection
- 修复 auto-layout 问题
- 生成或更新组件 variant。

**很重要的一点：**官方建议 `use_figma` 最好配合 `figma-use skill` 使用，这样 agent 在写 Figma 时更稳定、更符合推荐流程。

#### 4.3.2 `search_design_system`

这个工具会跨所有已连接的设计库，搜索匹配文本查询的 **组件、变量、样式**。它的目标非常明确：让 agent 先复用已有设计系统资产，而不是凭空创建新东西。

**典型用途：**

- 找现有 Button / Card / Empty State 组件
- 找主色变量、间距 token
- 找 icon style、text style。

#### 4.3.3 `create_new_file`

这个工具会在你的 drafts 里创建新的空白 Figma Design 或 FigJam 文件。如果你属于多个计划/组织，它还会让你选要创建到哪个 team / org。

**典型用途：**

- 新建 “Homepage Redesign”
- 新建项目规划用的 FigJam board
- 作为 `use_figma` 的前置步骤，先开一个干净文件再写内容。

#### 4.3.4 `generate_figma_design`

这是 remote only 工具，而且官方明确注明 **只在部分 MCP 客户端中可用**。它的作用是把你的 live UI 发送到 Figma，转成可编辑的 design layers，可以进新文件、已有文件或剪贴板。

**它对应的是“Code to Canvas”场景：**不是从 Figma 读设计去写代码，而是把浏览器里的真实页面反向送进 Figma。

**适用场景：**

- 把生产环境 / staging / localhost 的 UI 采集回 Figma
- 做设计回溯、审查、对齐
- 让产品/设计师直接在可编辑 Figma 层上讨论代码产物。

## 4.4 规则、身份和图表类

### 4.4.1 create\_design\_system\_rules

这个工具用于生成一份 **rule file**，给 agent 提供“你们项目该怎样把设计翻译成前端代码”的规则上下文。官方明确建议生成结果应保存到正确的 **rules/** 或 **instructions/** 路径，便于 agent 在后续代码生成时读取。

它很适合“Figma + Spec/规则文档 + AI 开发”的团队场景。

因为它本质上是在解决一个关键问题：让 AI 不只看设计，还知道你们项目的技术栈、设计系统约束、组件复用规则和代码风格。

### 4.4.2 whoami

这是 remote only 工具，用来返回当前连接到 Figma 的身份信息，包括邮箱、所属 plans、每个 plan 的 seat type。

适用场景：

- 检查是否授权成功
- 检查当前账号挂在哪些团队/组织
- 排查为什么有些写操作不能执行。

### 4.4.3 generate\_diagram

这是 remote only 工具，用于把 Mermaid 转成 可编辑的 FigJam diagram。而且官方明确说：你不一定要手写 Mermaid，也可以直接自然语言描述，agent 会自己生成 Mermaid 再调用工具。支持 flowchart、gantt、state、sequence 等类型。

适用场景：

- 认证流程图
- 支付时序图
- 项目排期图
- 状态机图。

## 5. 工具之间的关系

用一句话概括各类工具的层级关系：

- **get\_design\_context / get\_variable\_defs / get\_screenshot/get\_metadata**：是“读取设计上上下文”的核心层。
- **get\_code\_connect\_map / add\_code\_connect\_map / get\_code\_connect\_suggestions / send\_code\_connect\_mappings / get\_context\_for\_code\_connect**：是“把 Figma 组件和真实代码组件打通”的连接层。
- **use\_figma / create\_new\_file / search\_design\_system / generate\_figma\_design**：是“写回画布 / 创建设计 / 复用设计系统”的执行层。
- **create\_design\_system\_rules**：是“给 agent 建立团队规则”的约束层。

- `get_figjam / generate_diagram` : 是“流程图 / 白板 / 架构讨论”的辅助层。

## 6. Figma MCP 最适合的真实 workflow

---

### 6.1 workflow 1: 设计转代码（最典型）

核心链路：

1. 提供 file / frame / layer 链接
2. `get_design_context` 读取结构化设计
3. `get_variable_defs` 补充 token 信息
4. `get_screenshot` 保留视觉保真度
5. 如有 Code Connect, 再用映射让 AI 复用真实组件
6. AI 输出 Vue / React / HTML 等目标代码。

### 6.2 workflow 2: 设计系统对齐

若重点是“AI 生成代码贴近项目现有组件与 token”，需组合以下工具：

- `get_variable_defs`
- `search_design_system`
- `get_code_connect_map`
- `create_design_system_rules`

这样模型不仅知道“设计长什么样”，还知道“该用哪个组件、哪个 token、哪些规则”。

### 6.3 workflow 3: 代码反推回 Figma

若已有前端页面，希望把 live UI 回灌进 Figma，重点使用 `generate_figma_design`。适合做设计回收、现网对齐、把工程产物带回设计协作。

### 6.4 workflow 4: AI 直接修改 Figma

若希望 AI 在 Figma 里直接生成 frame、修 auto-layout、补 variables、生成 variants，重点使用 remote 下的 `use_figma`，最好搭配技能包使用。

## 7. 核心结论与实操建议

---

### 7.1 核心结论

若目标是“把 Figma 设计图直接喂给 AI，让 AI 基于真实设计上下文生成前端代码，且尽可能贴近设计系统和现有组件”，推荐理解框架：

1. 首选 remote MCP：官方明确推荐，功能最广、更新最快。

2. 核心读图工具不是截图，而是结构化设计上下文：`get_design_context`、`get_variable_defs`、`get_screenshot` 三者组合，是高质量 design-to-code 的关键。
3. 想让 AI 复用项目现有组件，必须接入 Code Connect：否则模型仍会“猜组件”，Figma 官方也强调，不用 Code Connect 时，模型对设计系统的理解有限。
4. `create_design_system_rules` \*\*想把项目规则落地，很重要\*\*：可将“设计规范、组件复用策略、代码风格、技术栈约束”转成 AI 可执行规则。

## 7.2 实操建议（适配 Vue3 + Spec Kit + AI 开发流程）

优先按以下顺序落地，更稳妥高效：

1. 先打通 remote MCP + `get_design_context` + `get_variable_defs` + `get_screenshot`，让 AI 能稳定读取真实设计。
2. 再补充 Code Connect，把设计组件和代码组件映射起来。
3. 最后引入 `create_design_system_rules`，把项目规范和技术约束固化给 agent。

避免一上来就追求“设计稿直接全自动生成完整项目”，逐步落地更易出效果。

## 8. GLips / Figma-Context-MCP（现名 Framelink MCP for Figma）

相关仓库链接 <https://github.com/GLips/Figma-Context-MCP?tab=readme-ov-file> 可正常访问；

### 8.1 核心定位

这个仓库本质上是一个社区开源 MCP server，现在在 README 中的产品名称是 Framelink MCP for Figma。它的定位非常明确：把 Figma 里的设计数据提供给 Cursor 等 AI 编码工具，让 agent 不再只看截图，而是读取经过整理后的布局和样式信息，用来更准确地“一次成型”实现界面。

仓库 README 直接强调，它的目标是让 coding agent 获取 Figma 数据后，比“贴截图给 AI”这种方式更适合做 UI 落地。

和 Figma 官方 MCP 不同，它不是 Figma 官方托管的 remote MCP，而是一个基于 Figma API 的第三方开源实现。它更像“面向代码生成场景的轻量 Figma 数据摄取层”，重点是把原始 Figma API 响应简化、翻译、压缩后再交给模型，而非提供官方那整套读写画布、Code Connect、设计系统搜索等完整能力。

### 8.2 与 Figma 官方 MCP 的关系

两者属于两条不同路线，核心区别如下：

- Figma 官方 MCP：官方产品能力，优先推荐 remote MCP，链接驱动，可提供 components、variables 等结构化上下文，支持写回画布、生成变量、Code Connect 映射等完整工作流。
- GLips / Framelink MCP：社区工程化实现，核心目标更聚焦一把 Figma 设计压缩成更适合 coding agent 消费的上下文。其 CLAUDE.md 明确项目哲学：职责是 “ingesting designs for AI consumption”（为 AI 摄取设计数据），而非代码生成器本身，也不负责 CMS 同步、复杂第三方集成等扩展范围。

选型建议：

- 若需 **官方首选、能力最全、长期方向最正统** 的方案，优先选 **Figma remote MCP**。
- 若需 **本地可控、轻量、专门服务于“Figma → Coding Agent → 代码”** 的开源接入层，GLips 仓库很实用。

## 8.3 核心设计思想

该项目最核心的价值，不是“能连接 Figma”，而是会 **先处理上下文，再交给模型**。README 明确写到：它在把 Figma API 的结果返回给模型之前，会做 **简化和翻译**，只保留最相关的 layout 和 styling 信息，以减少无效上下文，提升模型输出的准确性和相关性。

官方文档补充说明，Framelink MCP 会把收到的 Figma API 数据压缩将近 **90%**；即便如此，复杂设计仍可能让 agent 上下文过载，因此建议 **一次只处理一个 frame 或 group**。

这与 Figma 官方 remote MCP 思路不同：官方更强调“结构化设计上下文 + 丰富工具 + skills”；而 Framelink 更强调“**把设计数据裁剪成 AI 更容易处理的格式**”。

## 8.4 核心工具

该 MCP 的核心工具较少，主要为以下两个，专注于设计数据摄取和图片导出：

### 8.4.1 get\_figma\_data

主工具，用于 **获取并简化 Figma 设计数据**。项目架构文档定义为 “Fetches and simplifies Figma design data”。实际工作流中，用户把 Figma 文件、frame 或 group 的链接贴给 agent，agent 调用该工具读取并整理设计信息，再据此生成代码。

默认输出格式是YAML（作者推荐，更省 token），也可通过 `--json` 切换为 JSON 格式。

### 8.4.2 download\_figma\_images

配套工具，用于 **下载 Figma 中的图片资源**。架构文档定义为 “Downloads images from Figma”，配置文档说明它会把图片写入本地目录，可通过 `--image-dir` 指定允许写入的根目录，也可通过 `--skip-image-downloads` 禁用该工具。

## 8.5 技术结构

内部链路大致如下：

1. MCP tools 定义工具入口
2. Figma service 调用 Figma REST API
3. extractor system 把原始响应转换成更适合模型消费的数据
4. transformers 分别处理 layout、style、effects、text、component 等信息。

可见，它并非简单透传 Figma API 响应，而是专门做了“抽取-转换-压缩”层，这也是其在 Cursor 等 coding agent 场景中表现更稳定的关键。

## 8.6 实际使用方式

Framelink 文档推荐用法：先配置好 MCP，然后右键某个 frame 或 group，复制 selection link，把链接贴给 agent，再让 agent 实现该设计。

特别提醒：不要一次性喂整个大页面，应“one section at a time”（一次一个区域），因为即便做了压缩，复杂设计依然可能让 agent 上下文过载。

该用法与“复制文件/frame/layer 链接贴给 agent”的习惯完全契合，适用于官方 remote MCP 和 Framelink MCP 两种方案。

## 8.7 优势

- 比截图驱动更强：交给模型的是经过整理的设计数据，而非纯视觉像素，生成代码更准确。
- 上下文更轻：核心竞争力是把上下文做薄，让 Cursor 等 agent 更容易直接落地代码。
- 更适合“代码实现导向”的工作流：若目标是“根据 Figma 快速生成 Vue/React 页面、组件结构、样式骨架”，使用体验更顺手。

## 8.8 局限和风险

- 能力面较窄：相较于官方 MCP，缺少读取 variables、搜索设计系统、Code Connect、写回 Figma canvas、生成 FigJam diagram 等功能，核心仅两个工具。
- 工程稳定性有边界：社区 issue 中存在图片批量下载失败、大设计导致 agent 反复调用 `get_figma_data` 等问题（单张下载可作为临时解决方案）。
- Figma Make 支持不完整：有 issue 报告，用 Make project link 调用 `get_figma_data` 会返回 400 错误；转成 Design project 后，仅能拿到静态 UI，不包含交互逻辑。
- 版本元数据不同步：GitHub Releases 已更新至 v0.9.0（2026-04-09），但 main 分支 raw `package.json` 显示 0.8.1，raw `server.json` 显示 0.6.4，集成时需用 `npm run figma-developer-mcp --version` 确认实际版本。
- 安全风险：版本  $\leq 0.6.2$  存在高危命令注入问题，0.6.3 已修复，不建议使用较老版本。

## 8.9 定位评价

GLips/Figma-Context-MCP（Framelink MCP）是一个面向 AI 编码工具的社区开源 Figma MCP 实现。它的强项不是工具面最全，而是把 Figma API 的原始设计数据压缩、简化为更适合 coding agent 消费的上下文，因此特别适合“Figma → Cursor/Codex → 前端代码”的快速落地链路。

边界判断：如果目标是官方首选、长期演进能力最完整、并且希望支持 variables / Code Connect / design system search / write-to-canvas / FigJam diagram 等完整工作流，应优先使用 Figma 官方 remote MCP；如果目标是本地快速接入、轻量读取设计上下文、让 coding agent 更稳定地产生前端代码，Framelink MCP 是一个很实用的社区方案。

## 8.10 场景适配建议

- 做“官方方案研究报告”：以 Figma 官方 remote MCP 为主线。
- 做“工程落地和本地接入实验”：把 GLips / Framelink MCP 作为社区实现案例补充。
- 做“Vue3 + Spec Kit + AI 编码”链路验证：Framelink MCP 适合第一阶段原型，因其简单、直指 coding workflow（设计→代码）。



# Codex、AGENTS、Spec Kit 协同流程规范及实践

## 1 先定角色：Codex、AGENTS、Spec Kit 各自管什么

在本协同流程中，三者建议按以下方式明确分工，确保职责清晰、流程顺畅。

### 1.1 Codex Plan 模式：解决“这次到底怎么做”

Plan mode 用于让 Codex 先收集上下文、提出澄清问题、形成执行方案，再进入编码环节。官方明确说明：复杂任务优先使用 Plan mode，可通过 `/plan` 或快捷键切换。（[developers.openai.com](https://developers.openai.com)）

因此，它适合处理以下场景：

- 本次功能的边界范围是什么
- 哪些需求存在模糊点，需要进一步澄清
- 优先实现 MVP（最小可行产品）还是全量功能
- 是否需要拆分阶段逐步实现

### 1.2 AGENTS.md：解决“这个仓库长期怎么做”

OpenAI 官方建议将 `AGENTS.md` 作为 agent 的 README 文件，用于记录仓库长期遵循的规则，包括：

- 仓库结构规范
- 启动、构建、测试、lint 等常用命令
- 项目工程约定
- 开发禁止事项
- 功能“完成”的定义及验证标准。（[developers.openai.com](https://developers.openai.com)）

同时，它支持分层加载，优先级从高到低依次为：`~/.codex/AGENTS.md`（全局级）、仓库根目录 `AGENTS.md`（项目级）、更靠近当前目录的局部规则文件（局部级），越靠近当前工作目录的规则，优先级越高。（[developers.openai.com](https://developers.openai.com)）

### 1.3 Spec Kit：解决“从需求到实现的规范化工件”

Spec Kit 的定位并非单纯“编写 prompt”，而是将项目推进拆分为多个标准化阶段，逐步产出 spec（规格）、plan（计划）、tasks（任务）等规范化工件。官方 quickstart 给出的标准流程如下：

- constitution：制定项目核心原则
- specify：定义功能规格，重点强调“做什么（what）”和“为什么做（why）”，不涉及技术栈
- clarify：澄清需求中的模糊点，减少后续返工
- plan：确定技术方案与架构设计

- tasks: 拆解可执行的具体任务
- analyze: 交叉分析各工件的一致性与覆盖度
- implement: 按照任务清单执行实现。( [github.github.com](https://github.com) )

## 2 Figma URL 应该放哪里

Figma URL 的存放位置是本流程中易混淆的关键点，需根据其用途明确区分存放场景。

### 2.1 推荐结论

长期稳定的 Figma 入口，建议放入 **AGENTS.md**；当前 feature 的具体 frame / layer URL，不建议放在仓库级 **AGENTS.md**，应放在任务级上下文或功能级工件中。

具体说明：

- 放入 **AGENTS.md** 的内容（长期有效规则）：设计系统主文件链接、组件库链接、“前端实现以 Figma 为 UI 基准”的全局规则、使用 Figma MCP 的约定。这符合 OpenAI 对 **AGENTS.md** 作为“长期有效指导”的定义。( [developers.openai.com](https://developers.openai.com) )
- 不放入 **AGENTS.md** 的内容（一次性、任务级上下文）：当前 feature 的具体 frame / layer URL，更适合放在当前 Codex prompt、**/speckit.specify**、**/speckit.plan** 或当前 feature 的 spec 文档中。这基于 OpenAI 对 **AGENTS.md** 作为“durable guidance”（持久化指导）的要求：一次性上下文不应长期污染仓库全局规则。( [developers.openai.com](https://developers.openai.com) )

### 2.2 一个比较稳的写法

在 **AGENTS.md** 中仅保留长期有效的 Figma 相关规则，示例如下：

```
1 # 设计参考
2 - 前端实现必须遵循Figma设计系统及交互模式。
3 - 主要设计系统文件：[稳定的Figma设计系统链接]
4 - 需要设计上下文时使用Figma MCP。
5 - 功能特定的框架链接应提供在当前任务提示或规格工件中，不永久存储于此。
```

## 3 完整流程：用“AI 学习计划助手”跑一遍

以下结合“AI 学习计划助手”项目，给出一套完整可执行的协同链路，明确各环节操作与规范。

### 3.1 第 0 步：初始化仓库环境

先在项目目录中准备 Codex 和 Spec Kit 两类基础设施，确保流程可正常启动。

### 3.1.1 Codex 侧

- 开启 `Plan mode`，复杂任务优先使用该模式
- 使用 `/init` 生成初始 `AGENTS.md`，再根据项目实际需求手工完善。OpenAI 官方明确说明 `/init` 可快速生成 starter 版本的 `AGENTS.md`。（[developers.openai.com](https://developers.openai.com)）

### 3.1.2 Spec Kit 侧

Spec Kit 官方说明，大多数 agent 暴露的是 `/speckit.*` 系列命令；Codex CLI 在 `skills mode` 下使用的是 `$speckit-\*`。（[github.com](https://github.com)）

具体对应关系：

- Claude / 多数聊天 agent：`/speckit.constitution`、`/speckit.specify` 等
- Codex skills mode：`$speckit-constitution`、`$speckit-specify` 等

## 3.2 第 1 步：先写 `AGENTS.md`

`AGENTS.md` 聚焦仓库长期规则，不包含具体业务需求，核心是明确“仓库如何长期稳定运行”。

### 3.2.1 适合放进去的内容

- 技术栈：Vue 3 / Vite / Tailwind / local storage
- 常用命令：`npm run dev`、`npm run lint`、`npm run build`
- 工程约束：优先复用组件；不随意修改目录结构
- 验收标准：改动后必须运行 `lint/build` 检查
- 设计约束：严格遵守 Figma 交互和样式规范
- Figma 入口：设计系统主链接、组件库链接
- 数据约束：本地持久化，初始阶段不使用数据库

此类内容正是 OpenAI 官方建议放在 `AGENTS.md` 中的核心内容。（[developers.openai.com](https://developers.openai.com)）

### 3.2.2 示例

```
1 # AGENTS.md
2
3 # 项目概述
4 - 这是一个Vue 3 + Vite应用。
5 - 用户计划和进度本地持久化；除非明确要求，否则不引入数据库。
6
7 # 工作约定
8 - 非简单功能优先使用Plan模式。
9 - 优先进行最小化、针对性修改，而非大范围重构。
10 - 添加新的生产依赖前需询问确认。
11
12 # 设计规则
13 - 严格遵循Figma设计系统及交互模式。
14 - 设计系统文件：[Figma设计系统链接]
15 - 实现UI时，如需结构化设计上下文，使用Figma MCP。
```

```
16 - 功能特定的框架链接属于任务提示或规格工件，不存放于此。
17
18 # 必要检查
19 - 开发: `npm run dev`
20 - 代码检查: `npm run lint`
21 - 构建: `npm run build`
22
23 # 完成标准
24 - 实现所需功能
25 - UI与Figma设计意图一致
26 - 相关检查均通过
```

### 3.3 第 2 步: **\$speckit-constitution** 建立宪法

这一步聚焦**全局研发原则**，不涉及具体功能实现，核心是定义项目的开发准则，指导后续所有开发环节。

Spec Kit 官方说明，constitution 用于创建项目的 governing principles（指导原则）和 development guidelines（开发指南），贯穿项目全生命周期。（[github.com](https://github.com)）

#### 3.3.1 本项目适合的宪法内容

- Figma-first UI fidelity（UI 优先遵循 Figma 设计）
- 本地持久化优先
- 需求先 clarify 再 plan（先澄清需求再制定计划）
- MVP 分阶段交付
- 变更必须可验证
- 前端交互一致性优先于炫技

#### 3.3.2 示例 prompt

```
1 $speckit-constitution
2 本项目遵循Figma优先原则。UI及交互行为必须与已确认的Figma设计保持一致。
3 所有数据持久化均以本地优先，初始阶段不使用数据库。
4 我们倾向于先交付可运行的MVP（最小可行产品），再开发次要功能。
5 每一项功能变更都必须通过明确的验证步骤进行验证。
6 优先选择简单架构和最少依赖。
```

### 3.4 第 3 步: **\$speckit-specify** 只写 what 和 why

Spec Kit 官方明确强调: **/speckit.specify** 需重点说明“要做什么（what）、为什么做（why）”，**不建议在该阶段讨论技术栈**，避免过早锁定实现方式。（[github.github.com](https://github.com)）

### 3.4.1 正确写法建议

不建议写的内容（技术栈相关）：

- Vue3、localStorage、OpenAI API 等技术细节
- 组件拆分、代码实现方式

建议写的内容（需求相关）：

- 目标用户是谁
- 用户想解决什么核心问题
- 需要哪些核心功能
- MVP 包含的核心能力
- 这些功能对用户的价值（为什么重要）

### 3.4.2 示例 prompt

```
1 $speckit-specify
2 开发一款面向备考学生的AI学习计划助手。
3 该产品应帮助用户根据目标考试、可用时间、当前水平和偏好的学习强度创建学习计划。
4 用户应能够创建多个学习计划、查看进度并跟踪每日任务。
5 仪表盘应帮助用户快速了解整体进度和当前优先级。
6 此功能很重要，因为用户通常难以将模糊的学习目标转化为可执行的计划。
7 所有面向用户的界面均以已确认的Figma设计为UI参考。
8 当前框架参考：
9 - 仪表盘：[框架链接]
10 - 创建计划流程：[框架链接]
11 - 计划详情页：[框架链接]
```

注：此处放置具体 frame URL 是合理的，因为这属于当前 feature 的上下文，而非仓库级长期规则。

## 3.5 第 4 步：\$speckit-clarify 把模糊点问透

Spec Kit 官方建议，在制定 plan 之前先运行 clarify 环节，明确需求模糊点，减少后续计划和实现阶段的返工。（[github.github.com](https://github.com)）

针对“AI 学习计划助手”项目，以下是典型的需求模糊点，需重点澄清：

### 3.5.1 重点澄清方向

- 仪表盘默认显示视角（如默认显示所有计划、当前活跃计划等）
- 多计划并行时的优先级逻辑（如按创建时间、重要程度排序）
- AI 生成计划后，用户是否允许手动修改
- 学习任务的完成状态如何定义（如点击完成按钮、满足特定条件）
- 当 Figma 设计与文档说明冲突时，以哪个为准
- 是否支持离线使用，离线数据如何同步
- 用户首次使用是否需要引导流程

### 3.5.2 主动补充澄清方向的示例

```
1 $speckit-clarify
2 重点关注仪表盘默认显示规则、计划优先级、任务完成状态、AI计划可编辑性，以及Figma与书面规格的冲突解决方式。
```

该写法可引导 agent 聚焦核心模糊点，提升澄清效率。

## 3.6 第 5 步：\$speckit-plan 进入技术实现

进入 plan 环节后，方可明确技术栈、架构设计等“如何实现（how）”的内容，这与 Spec Kit 官方流程要求一致。（[github.github.com](https://github.com)）

### 3.6.1 本环节需明确的核心内容

- 前端技术栈：Vue 3 + Vite
- 模型调用：OpenAI API
- API key 来源：全局环境变量（避免硬编码）
- 本地持久化方案：localStorage / IndexedDB
- 页面路由设计
- 核心数据模型（如计划、任务、进度等）
- API 接口约定
- 是否使用 mock 数据，是否真实调用模型
- 状态管理方式（如 Pinia、Vuex 或简单响应式）
- 数据校验、异常处理方式

### 3.6.2 示例 prompt

```
1 $speckit-plan
2 前端使用Vue 3 + Vite开发。
3 模型调用必须使用OpenAI API，且API密钥需来自环境变量。
4 学习计划、任务和进度必须本地持久化，不使用数据库。
5 布局和交互严格遵循已确认的Figma设计。
6 当Figma设计与规格说明冲突时，实现细节以书面规格为准。
7 初始实现应包含：
8 - 创建学习计划流程
9 - 使用真实模型响应生成AI任务分解
10 - 仪表盘
11 - 计划详情与进度跟踪
12 - 优先采用简单架构，减少依赖。
```

### 3.6.3 本环节常见产物

根据 Spec Kit 官方示例，feature 目录下通常会生成以下工件：

- **spec.md**：功能规格文档
- **plan.md**：技术实现计划

- `research.md`：技术调研文档
- `data-model.md`：数据模型文档
- `quickstart.md`：快速启动说明
- `contracts/`：接口约定目录（如 API 规范等）。([github.com](https://github.com))

注：原文中提到的 `contracts/openapi.yaml` 是常见的接口约定形态，但 Spec Kit 官方示例中展示的是 `contracts/api-spec.json` 及其他约定文件，因此更稳妥的表述是：`contracts` 目录下会存放接口约定工件，具体格式取决于项目模板。([github.com](https://github.com))

### 3.7 第 6 步：\$speckit-checklist 检查 spec 是否完整

Spec Kit 官方对 checklist 的定义是：生成自定义质量清单，用于验证需求的完整性、清晰度和一致性，是“需求验收前检查”的关键环节。([github.com](https://github.com))

#### 3.7.1 检查重点方向

- 所有功能点是否可测试、可验证
- 是否存在未澄清的术语或需求
- 是否有不可实现的隐含要求
- UI 各状态（正常态、空态、错误态等）是否完整
- 异常流程（如网络错误、数据异常）是否缺失

#### 3.7.2 示例 prompt

```
1 $speckit-checklist
2 生成一份检查清单，用于验证本功能规格是否完整、可测试、可实现，且符合Figma驱动的UI需求。
```

### 3.8 第 7 步：\$speckit-tasks 任务拆解

Spec Kit 官方说明，`/speckit.tasks` 会根据 `implementation plan`（技术实现计划），生成可执行的任务清单，为后续 `implement` 环节提供明确依据。([github.github.com](https://github.com))

#### 3.8.1 本项目合理的任务拆解示例

1. 初始化 Vue3 项目和基础目录结构
2. 建立本地数据模型（定义计划、任务、进度等数据结构）
3. 实现“创建学习计划”表单（含用户输入、校验逻辑）
4. 集成 OpenAI API，实现 AI 生成任务规划功能
5. 开发计划详情页，展示任务列表及进度
6. 实现仪表盘聚合逻辑（展示全局进度、优先级任务等）
7. 集成本地持久化，实现数据存储与读取
8. 开发进度更新与回显功能（如任务完成状态切换）
9. 补充基础校验和异常态处理（如空输入、API 调用失败）
10. 运行 lint/build 检查，并进行手工验证

注：任务拆解是 `implement` 环节的核心前提，需确保任务有序、可执行，且覆盖所有 `plan` 中定义的实现内容。

### 3.9 第 8 步：\$speckit-analyze 交叉分析

Spec Kit 官方将 `analyze` 定义为 **cross-artifact consistency & coverage analysis**（跨工件一致性与覆盖度分析），建议在 `tasks` 之后、`implement` 之前运行，用于发现潜在问题。（[github.com](https://github.com)）

#### 3.9.1 本环节适合发现的问题

- `spec` 中定义的需求，`tasks` 未覆盖
- `plan` 中设计的架构（如状态管理），`tasks` 未落地
- Figma 要求的空态、错误态，`spec` 中遗漏
- 数据结构与页面流程存在冲突

#### 3.9.2 示例 prompt

```
1 $speckit-analyze
2 分析宪法、规格、计划、检查清单和任务之间的一致性和覆盖度。
3 重点关注缺失的状态、未覆盖的需求，以及数据模型、UI流程和实现任务之间的冲突。
```

### 3.10 第 9 步：\$speckit-implement 实现前先清上下文

原文提到“先清空上下文”的做法，是合理的工程习惯。进入 `implement` 环节时，需确保 `agent` 基于最终确认的工件执行，避免前期探索性对话上下文干扰，提升执行准确性。

Spec Kit 官方对 `implement` 的说明如下：

- 检查 `constitution`、`spec`、`plan`、`tasks` 等前置工件是否齐全
- 解析 `tasks.md` 中的任务清单
- 按任务依赖顺序执行实现
- 遵循任务计划中的 TDD（测试驱动开发）思路
- 实时提供进度更新，处理执行过程中的错误。（[github.com](https://github.com)）

#### 3.10.1 实施建议

在执行 `$speckit-implement` 前，仅保留以下核心上下文，确保 `agent` 聚焦执行：

- `AGENTS.md`（仓库长期规则）
- `constitution`（项目研发原则）
- `spec`（功能规格）
- `plan`（技术实现计划）
- `tasks`（任务清单）
- 当前 `feature` 所需的 Figma frame URL

## 4 如果不用 Spec Kit，怎么轻量化

若项目规模较小、需求简单，无需使用 Spec Kit 的完整流程，可采用轻量化方案，更贴合小项目、快速原型的开发需求。OpenAI 官方对 Codex 的轻量化使用有明确建议：

- 复杂任务先用 Plan mode
- 把可复用约束写进 `AGENTS.md`
- 把重复工作流升级成 Skills。([developers.openai.com](https://developers.openai.com))

### 4.1 轻量流程

1. 使用 `/init` 生成初始 `AGENTS.md`，并根据项目需求完善
2. 使用 `/plan` 和 Codex 澄清需求、明确边界、确认 Figma 入口
3. 让 Codex 直接生成一份 `PLANS.md` 或 feature plan（功能实现计划）
4. 将重复出现的工作流（如代码校验、测试）做成 Skills，提升复用效率
5. 进入编码环节，直接执行实现

### 4.2 适合场景

- 小功能开发、快速原型验证
- 单人项目，无需复杂的工件管理
- 不想维护大量 spec 相关工件，追求高效迭代

### 4.3 不足

- 需求可追踪性弱，后期难以追溯需求来源与实现逻辑
- 需求变更多时，容易出现需求漂移，难以保持一致性
- 团队协作时，缺乏统一的规范化工件，沟通成本较高

## 5 AGENTS 设计详解

“给 Codex 的项目级行为规范文件”——它负责定义默认工作方式；写法上要短、硬、可执行、可验证、可分层覆盖；真正复杂的复用流程不要继续堆在这里，而应该交给 Skills。

### 5.1 AGENTS.md 是什么

对 Codex 而言，`AGENTS.md` 并非普通文档，而是**持久化项目指令**。Codex CLI 会自动发现这些文件，并将其注入到工作上下文；在官方 Prompting Guide 中，这些内容会以独立的 `user-role message` 形式，出现在用户提示之前。

这意味着它最适合存放“**每次工作都应生效**”的规则，例如：

- 项目如何启动、构建、测试
- 修改代码后必须执行哪些检查

- 哪些目录、文件有特殊约束（如禁止修改）
- 哪些操作需要先确认再执行（如新增依赖）
- PR、文档、测试的最低要求

它不适合堆放一次性的任务说明（如“本次需修改某个页面的按钮样式”）。官方明确将 **AGENTS.md** 定义为**随仓库一起迭代的 durable project guidance**（持久化项目指导）；若需实现更复杂、可复用、带脚本/参考资料的流程，官方更推荐使用 **Skills**。此外，自定义 prompts 已被标记为 **deprecated**（废弃），建议迁移到 **Skills**。

### 5.1.1 明确 **AGENTS.md** 和 Spec-kit 各工件的职责分工

- **AGENTS.md**：仅定义 Codex 的工作方式（仓库级长期规则）
- **.specify/memory/constitution.md**：仅定义项目长期研发原则
- **specs/.../spec.md**：定义某个具体功能“要做什么、为什么做”
- **specs/.../plan.md**：定义该功能“如何实现”
- **specs/.../tasks.md**：定义该功能的具体任务拆解清单

## 5.2 Codex 如何发现并合并这些文件

Codex 启动时，会自动构建一条“指令链”，按以下规则加载并合并 **AGENTS.md** 相关文件：

1. **全局层**：优先读取 **~/.codex/AGENTS.override.md**（全局临时覆盖规则），若不存在则读取 **~/.codex/AGENTS.md**（全局长期规则）；若设置了 **CODEX\_HOME** 环境变量，则从该变量指定的目录读取。
2. **项目层**：从仓库根目录开始，逐级遍历到当前工作目录；每一级目录都按“**AGENTS.override.md** → **AGENTS.md** → **project\_doc\_fallback\_filenames** 中定义的备用文件名”的顺序查找，每级目录最多加载一个文件。
3. **合并顺序**：按“全局层 → 项目根目录 → 子目录”的顺序拼接文件内容，**越靠近当前工作目录的规则，优先级越高**（后续规则会覆盖前置规则）。
4. **大小限制**：空文件会被自动忽略，所有合并文件的总字节数默认上限为 **project\_doc\_max\_bytes = 32 KiB**（可通过配置修改）。

官方补充细节：**每个被发现的文件都会作为一条单独的 user-role message 注入上下文**，消息头类似 **# AGENTS.md instructions for <directory>**。这也是为什么分层编写规则，比将所有规则塞进根目录一个大文件更稳妥、更易维护。

## 5.3 设计规范：一份好的 **AGENTS.md** 应该写什么

结合 OpenAI 官方文档、官方示例及 **openai/codex** 仓库自身的 **AGENTS.md** 实践，一份有效的 **AGENTS.md** 通常包含以下四类核心信息。

### 5.3.1 环境与入口

告诉 Codex**从哪里开始工作**，明确项目的基础环境信息，例如：

- 使用的包管理器（如 pnpm、npm）
- 启动开发环境的具体命令
- monorepo 项目中，如何快速定位目标 package

- 哪个目录是核心开发模块（需重点关注）

官方示例中，这类内容通常归为“Dev environment tips”（开发环境提示），是 Codex 快速熟悉项目的关键。

### 5.3.2 验证与测试

这是 **AGENTS.md** 最核心的内容之一，需明确“修改代码后如何验证正确性”，例如：

- 修改代码后，必须运行哪些检查（如 lint、typecheck、test）
- 哪些检查可自动执行，哪些需要先征求用户确认
- 全量测试与局部测试的边界（如修改共享模块需跑全量测试）
- lint、typecheck、snapshot 等检查与 CI（持续集成）的对齐方式

OpenAI 自身的 **codex** 仓库，就在 **AGENTS.md** 中明确写了 Rust 改动后需运行 **just fmt**、项目级测试、何时跑全量测试、何时更新 schema / lockfile 等规则，可见“可执行的验证规则”是 **AGENTS** 的核心价值。

### 5.3.3 代码与架构约束

明确项目的代码规范与架构限制，避免 Codex 做出不符合项目规范的改动，例如：

- 哪些模块已饱和，不建议继续膨胀（需拆分新模块）
- 新功能优先放在新模块，避免污染原有核心代码
- 哪些 API 设计禁忌需要避免（如禁止暴露敏感接口）
- 哪些文件、环境变量、安全逻辑禁止修改（如密钥配置）

官方仓库示例甚至将“不要修改某些 sandbox 相关代码”“避免大模块”“新增 API 要同步 docs/”等规则写入，说明 **AGENTS.md** 不仅是代码风格说明，更是把长期反复出现的 review 意见固化下来，减少重复沟通成本。

### 5.3.4 交付与协作约束

明确项目的交付标准与协作规范，确保团队协作顺畅，例如：

- PR（合并请求）标题、描述的格式要求
- 修改功能行为时，需同步更新相关文档
- 新增依赖是否需要先征求团队确认
- 涉及安全、密钥、生产配置时的沟通要求

官方全局示例和仓库示例都在强调这一层，可见 **AGENTS.md** 也是团队协作规范的重要载体。

## 5.4 推荐写法原则

结合 OpenAI 官方材料抽象，编写 **AGENTS.md** 需遵守以下五条原则，确保规则实用、可执行：

1. **聚焦长期有效规则，不写一次性任务**：OpenAI 在 Codex best practices 中明确将 **AGENTS.md** 定义为“durable guidance”（持久化指导），一次性任务（如“修复某个 bug”“开发某个页面”）应写在 prompt 中，而非 **AGENTS.md**。

2. **短、准、可执行，拒绝空泛**：官方原话明确：一个“short, accurate **AGENTS.md**”（简短、准确）比一份“long file full of vague rules”（冗长、空泛）更有用；建议先写基础规则，再在观察到 Codex 反复犯同类错误时，逐步补充。
3. **优先写可执行的约定**：例如明确“修改 JavaScript 后总是运行 **npm test**”“优先用 **pnpm**”“新增生产依赖前先确认”，而非“请写高质量代码”这类无约束力的表述。官方示例均采用“命令明确、动作明确、边界明确”的风格。
4. **通用规则与局部规则分层放置**：官方推荐将个人长期偏好放在 **~/.codex/AGENTS.md**（全局级），团队共享规范放在仓库根目录（项目级），子目录有特殊规则则单独放置 **AGENTS.override.md** 或 **AGENTS.md**（局部级），比单一文件更易维护、优先级更清晰。
5. **重复流程升级为 Skills，不堆进 AGENTS.md**：OpenAI 明确将 **AGENTS.md** 定位为持久化指导，而 Skills 定位为“可复用工作流”的载体。例如“发布流程”“数据库迁移流程”等多步骤 SOP，长期看更适合做成 Skills，避免 **AGENTS.md** 过于冗长。

## 5.5 通用项目模板（可直接修改使用）

以下模板基于官方规范归纳，适用于 Vue / React / Node / monorepo 项目，可根据项目实际需求调整：

```
1 # AGENTS.md
2
3 # 项目范围
4 - 本文件适用于整个仓库，除非更深层的AGENTS.md对其进行覆盖。
5
6 # 工作规则
7 - 优先进行最小化、局部化修改，而非大范围重构。
8 - 非必要不添加新依赖。
9 - 修改生产配置、密钥、CI或部署文件前需询问确认。
10
11 # 开发环境
12 - 包管理器：pnpm
13 - 主应用：apps/web
14 - 主API：apps/server
15 - 对于monorepo包查找，编辑前请先查看每个package.json的name字段。
16
17 # 常用命令
18 - 安装：pnpm install
19 - 开发：pnpm dev
20 - 代码检查：pnpm lint
21 - 类型检查：pnpm typecheck
22 - 测试：pnpm test
23
24 # 代码变更策略
25 - 优先复用现有模式，再引入新的抽象概念。
26 - 尽量减少公共API的变更，且变更需明确。
27 - 修改共享工具时，需检查所有受影响的调用者。
28
29 # 测试策略
```

```
30 - 代码变更后，需运行代码检查和类型检查。
31 - 功能行为变更时，需新增或更新测试用例。
32 - 优先运行针对性测试；仅当共享模块被修改时，才运行完整测试套件。
33
34 # 文档策略
35 - 若功能行为或公共API发生变更，需更新docs/目录下的文档。
36 - 若环境配置发生变更，需更新README或模块文档。
37
38 # 合并请求/交付要求
39 - 总结变更内容、变更原因及验证方式。
40 - 说明任何潜在风险、后续工作或有意跳过的检查项。
41
42 # 目录专项说明
43 - `apps/web`：重点关注UI响应式、可访问性及路由级性能。
44 - `apps/server`：除非明确要求，否则避免破坏schema的变更。
```

模板结构贴合官方示例与实践，涵盖：作用范围、工作规则、开发环境、常用命令、代码变更策略、测试策略、文档策略、交付策略、目录特化说明，可直接复用。

## 5.6 如何使用 AGENTS.md

### 1) 全局个人偏好（个人级）

将个人长期开发习惯写入全局 AGENTS.md，适用于所有个人项目：

```
1 ~/.codex/AGENTS.md
```

示例内容：总是先跑测试、偏好使用 `pnpm`、新增依赖前先确认等。若需临时覆盖全局规则，可创建 `~/.codex/AGENTS.override.md`，删除该文件即可恢复全局规则。

### 2) 项目团队规范（项目级）

在仓库根目录放置 AGENTS.md，写入团队共享约定：

```
1 AGENTS.md
```

示例内容：团队开发命令、测试规则、PR 要求、代码规范等，确保所有团队成员（含 Codex）遵循统一标准。

### 3) 模块专项规则（局部级）

在子目录（如特定服务、模块）放置 AGENTS.override.md 或该目录自身的 AGENTS.md，定义局部特殊规则：

```
1 services/payments/AGENTS.override.md
```

示例场景：支付模块的安全规则、后台管理模块的权限约束、前端应用的性能要求等，实现“全局统一、局部特殊”的规则管理。

## 4) 验证是否生效

官方推荐使用以下命令，验证 Codex 是否正确加载了 **AGENTS.md** 规则：

```
1 codex --ask-for-approval never "Summarize the current instructions."
2 codex --cd subdir --ask-for-approval never "Show which instruction files are active."
```

也可查看日志文件，确认加载情况：

```
1 ~/.codex/log/codex-tui.log # 或 session 日志文件（启用 session logging 后）
```

## 5) 已有项目不想改文件名

若项目已存在 **TEAM\_GUIDE.md** 等类似规则文件，无需修改文件名，可在 `~/.codex/config.toml` 中配置备用文件名：

```
1 project_doc_fallback_filenames = ["TEAM_GUIDE.md", ".agents.md"]
2 project_doc_max_bytes = 65536 # 可选：调整合并文件大小上限
```

配置后，Codex 会将这些文件也视为项目指令文件，按优先级加载。

## 5.7 AGENTS.md 和 Skills / Subagents 的区别

三者职责分工清晰，可简单总结为：

- **AGENTS.md**：项目级、长期有效、默认始终生效的规则，定义 Codex 的基础工作方式。
- **Skills**：某类任务的可复用 workflow，可包含 **SKILL.md**、脚本、参考资料，支持显式或隐式触发（如重复的测试、部署流程）。
- **Subagents**：任务复杂到需要并行执行、分角色分工、分模型配置时使用（如多模块并行开发、分角色负责测试与编码）。

实践建议流程：

1. 先用 **AGENTS.md** 固定仓库基础规则，确保 Codex 工作方式统一。
2. 再将重复出现的流程（如代码校验、文档更新）沉淀成 Skills，提升效率。
3. 任务复杂到需要分工协作时，再引入 Subagents。

## 5.8 常见误区

结合官方规范与实践，编写和使用 **AGENTS.md** 需避免以下误区：

### 1) 把它写成超长规范手册

- 官方有默认 32 KiB 上限；太长会被截断，更适合拆层或改成 Skills。

### 2) 只写“请写高质量代码”

- 这类话几乎没约束力。要改成“改完跑什么、别改什么、何时更新 docs、哪些目录有特殊要求”。官方示例和官方仓库都偏向这种可执行规则。

### 3) 把一次性任务说明塞进去

- 一次性需求应该写在 prompt 里；AGENTS.md 只放会反复使用的规则。官方把它定位为 durable guidance。

### 4) 全项目只放一个根文件

- 更好的方式是：根目录写共性，子目录写特性。因为 Codex 的规则是从根到当前目录逐级合并，近处覆盖远处。

本文将AI代码审查相关问题拆分为质量保障、开发者审核方法、Spec Kit产出不达预期时的修正方案三部分，结合官方文档做法，落地一套可执行流程。

**核心原则：**不要把“代码质量”寄托在最后一次code review上，而是把质量控制拆到“生成前、生成中、生成后”三段。

Spec Kit的价值，正是把这三段前移：先立原则，再写规格，再澄清，再做计划，再拆任务，最后才实现。GitHub官方流程就是 constitution → specify → clarify → checklist → plan → tasks → analyze → implement，并明确建议在 plan 前做 clarify，在 implement 前做 analyze，复杂项目分阶段实现。

## 6.1 如何保证 AI 生成代码的质量

最有效的方法，不是“让AI更聪明”，而是让它在更清晰的边界内工作。

### 6.1.1 生成前：先把“源头”做对

如果源头是模糊的，后面的代码再努力也会跑偏。OpenAI的Codex最佳实践建议，给代理的默认任务上下文至少包含四类信息：目标、上下文、约束、完成标准；这样能减少假设，让结果更容易审核。Codex还建议复杂任务先进入Plan mode，让代理先收集上下文、提澄清问题，再实施。

放到Spec Kit里，对应就是：

- constitution：定义项目原则和硬约束
- specify：定义what / why
- clarify：消除歧义
- checklist：检查规格完整性
- plan：给出技术方案
- tasks：拆成可验证小步

GitHub官方明确说明，/speckit.specify 应聚焦what和why，而不是tech stack；/speckit.clarify 推荐在 /speckit.plan 前执行，以减少下游返工；/speckit.checklist 用于验证需求完整性、清晰度和一致性。

**关键结论：**高质量AI代码的第一原则不是“写好实现prompt”，而是“把spec写对”。

### 6.1.2 生成中：把实现拆小，不要一口气生成整个功能

Spec Kit官方建议复杂项目分阶段实现，先完成核心功能，再逐步增加特性，避免一次性把agent的上下文压得太满。Codex也建议困难任务先plan，再逐步执行。

工程上最有效的做法是：

- 先让AI只完成一个小任务，比如“封装localStorage service”，而不是“把整个习惯打卡项目全做完”。
- 然后立即跑最小验证：单测、lint、build、手动操作。
- 验证通过后，再推进下一个task。

这样做有两个直接好处：第一，错误更容易定位；第二，偏差不会滚雪球。

### 6.1.3 生成后：用“验证链”而不是“主观感觉”判断质量

Spec Kit的 `tasks.md` 本身就会把任务拆成依赖顺序、文件路径、测试优先顺序和阶段检查点；`implement` 会先验证constitution/spec/plan/tasks是否齐全，再按任务顺序执行。官方还明确提醒，实现完成后要继续测试应用，并把浏览器控制台之类CLI看不到的错误再反馈给agent修复。

所以，AI代码质量要靠一条明确的验证链：

规格一致性 → 任务完成度 → 自动化测试 → 构建通过 → 运行态验证 → 人工评审

这比“看起来写得挺像样”要可靠得多。

## 6.2 开发者做代码审核时，正确的思路是什么

开发者review AI代码时，最常见的错误是只盯diff，不看spec。AI代码审核更好的顺序应该是：

### 6.2.1 第一步：先审“是不是做对了事”

先对照 `spec.md`、`plan.md`、`tasks.md` 看实现有没有偏题，而不是先看代码风格。因为在Spec-Driven Development里，spec是source of truth，GitHub官方博客也明确这么描述：spec是代码行为的契约，工具和AI会据此生成、测试和校验代码。

你先看四个问题：

1. 实现的是不是spec里的需求，而不是AI自己扩展出来的功能
2. 非功能约束有没有守住，比如“不要引入新依赖”“不要上状态管理库”
3. 边界条件有没有覆盖
4. 完成标准有没有达到

如果第一步没过，后面再看实现细节意义不大。

## 6.2.2 第二步：再审“技术路线对不对”

这一步对照的是 `plan.md`。你主要看：

- 架构是否过度设计
- 组件边界是否合理
- 数据流是否清晰
- 有没有引入不必要的抽象
- 有没有违背仓库级约束

Codex官方建议把repo layout、build/test/lint命令、工程约定、限制项、done的定义都写进 `AGENTS.md`，而且 `AGENTS.md` 会在Codex工作前自动读入。也就是说，review时不光要看代码，还要看它有没有遵守仓库级agent规则。

## 6.2.3 第三步：最后才审“代码本身写得好不好”

这一层才看具体实现：

- 命名是否清晰
- 逻辑是否容易理解
- 组件是否单一职责
- 错误处理是否完整
- 测试是否覆盖核心路径和边界情况
- 是否有明显重复代码
- 是否出现隐藏副作用

这里建议你吧review分成三类问题：

- **A类：需求偏差** 实现不符合spec，这种不能直接patch代码，要回到spec/plan/tasks层修。
- **B类：设计偏差** 功能对了，但方案不合适，比如状态放错层、耦合太重、抽象过头。这种通常要回到plan层修。
- **C类：实现缺陷** 方案没问题，只是有bug、漏判、测试不全、命名差。这种可以直接定点修代码。

这种分层很重要，因为它决定你应该改哪里，而不是一股脑让AI“重新生成一版”。

## 6.2.4 第四步：要求AI先解释，再改

如果你发现问题，不要直接说“重写”。更好的做法是先让AI回答三个问题：

1. 它是根据spec的哪一段做出当前实现的
2. 它为什么选这个方案
3. 哪个约束让它做了这个取舍

这样能快速判断问题出在spec、plan还是implement。OpenAI也强调，Codex更适合被当作“可持续配置和改进的队友”，而不是一次性助手。

### 6.2.5 第五步：把重复错误沉淀回规则

Codex最佳实践里有一个很实用的建议：**AGENTS.md** \*\*如果Codex连续犯同样的错，就让它做 retrospective，然后更新\*\* 。官方还建议 **AGENTS.md** 保持简短、实用，规则应来自真实摩擦，而不是空泛口号。

这意味着开发者review不只是“发现问题”，还要做一件更重要的事：把问题变成以后不会再犯的规则。

## 6.3 如果 Spec Kit 生成的代码没达到预期，应该怎么改

**核心原则：**不要一上来就改代码，先判断“错在spec、错在plan，还是只错在实现”。

### 6.3.1 情况 1：功能方向就错了

比如你想要“每天最多打卡一次”，结果AI做成了“可无限次打卡”；或者你要“删除习惯时保留历史”，它却做成了级联删除。

这类问题通常不是实现能力问题，而是spec没写清、clarify没补齐，或者checklist没卡住。正确做法不是“把实现patch一下”，而是：

1. 回到 **specify** 或 **clarify**
2. 把缺失的业务规则补进去
3. 重新跑 **checklist**
4. 必要时重做 **plan** 和 **tasks**
5. 再让 **implement** 只重做受影响的任务

因为source of truth是spec；如果只修代码，不修spec，下次agent还会按旧规则继续偏。GitHub官方把spec明确称为source of truth；Spec Kit也明确把 **clarify** 放在 **plan** 之前，就是为了减少这种返工。

### 6.3.2 情况 2：需求没错，但技术方案不合适

比如功能都对，但AI引入了没必要的抽象，或者用了不符合项目预期的状态管理方式。

这类问题通常出在 **plan.md**。正确做法是：

1. 修改 **constitution** 或 **AGENTS.md**，增加技术边界
2. 回到 **plan**，明确重写技术方案
3. 重新生成 **tasks**
4. 再实施受影响的任务

例如你可以明确写：

```
1 请不要修改 spec。
2 当前问题不是业务需求，而是技术方案过度设计。
3 请重做 implementation plan，要求：
4 - 保持 Vue 组件简单
5 - 不引入额外状态管理
6 - localStorage 只放在 service 层
7 - 先修正 plan，再重建 tasks，不要直接改代码
```

这类提示比“重写一版”更有效，因为它把问题定位到了正确层级。

### 6.3.3 情况 3：方案正确，只是代码实现有bug或质量差

这类问题最常见，比如：

- streak计算错一天
- localStorage没处理空值
- 某个组件职责混乱
- 测试漏了边界场景

这时就不需要回滚到spec。你可以直接针对具体task修：

```
1 当前 spec 和 plan 不变。
2 只修复 T4 的实现问题：
3 - streak 计算在跨天边界时错误
4 - 增加对应 Vitest 测试
5 - 不改动其他任务
6 修复后运行测试并总结原因
```

Spec Kit README明确说，`tasks.md` 提供的是 `/speckit.implement` 的路线图；也就是说，按 **task定点返修** 是最自然的修法。

## 6.4 一个最实用的“排障顺序”

如果Spec Kit产出不理想，你可以按下面顺序排查：

1. 先问：是spec不清，还是实现没跟上？

如果是“做了不该做的事”，先回 `specify/clarify/checklist`。如果是“做法不对”，先回 `plan`。如果是“做得不够好”，直接修 `tasks/implement`。

2. 再问：规则够不够具体？

Codex官方建议把repo layout、运行方式、测试命令、工程约定、限制项、done的定义写进 `AGENTS.md`。如果AI老是在相同地方偏航，通常说明这些约束没有真正写进仓库级规则。

3. 再问：任务是不是太大了？

Spec Kit官方建议复杂项目分阶段实现；如果一次让agent同时改太多文件、太多层，质量通常会下降。

#### 4. 再问：有没有做 `analyze`

Spec Kit README把 `/speckit.analyze` 定义为cross-artifact consistency & coverage analysis，并明确建议它放在 `tasks` 之后、`implement` 之前。很多“代码没达到预期”的问题，其实在 `implement`前就能在`analyze`阶段发现。

### 6.4.5 最后才考虑重生成

只有当前面几层都确认修不动时，才适合大范围重生成功能。否则很容易把一个局部问题，变成一次更大的不确定改动。

## 6.5 实际开发里固定使用的审核模板

---

每次review AI代码，都可以按这套顺序问：

#### 第一组：对不对

- 这次改动对应spec的哪一段
- 有没有违反constitution / AGENTS.md
- done的定义是否满足

#### 第二组：稳不稳

- 测试是否覆盖主路径和边界
- lint/build是否通过
- 运行态是否正常，有没有浏览器控制台错误
- 有没有引入多余复杂度

#### 第三组：值不值得沉淀

- 这次错误是偶发，还是会重复出现
- 是否应该把它写回AGENTS.md
- 是否应该拆成一个repo skill

Codex官方建议把重复工作变成skills，并且每个skill专注一个任务、写清输入输出、测试触发行为。

---

# 第七章 单Agent与多Agent实践

Anthropic 在《Building Effective AI Agents》里给出的主线其实非常明确：先从最简单的可行方案开始，只有当简单方案已经证明不够用时，再增加工作流和 Agent 复杂度。它把能力形态大致分成三层：增强型 LLM（加工具/检索/记忆）→ 工作流（链式、路由、并行、评审循环）→ 自主 Agent。其核心原则不是“技术越复杂越先进”，而是“复杂度必须能换来可证明的效果提升”。

## 1. 先把概念摆正：单Agent和多Agent不是“先进/落后”的关系

很多人容易陷入“多Agent比单Agent高级”的误区，实则二者是适配不同场景的方案，核心差异在于“任务拆分与协作模式”，而非技术层级的高低。

### 1.1 单Agent是什么

单Agent可以理解成：一个主体在一个主循环里感知环境、规划、调用工具、根据结果继续调整，直到完成任务或触发停止条件。Anthropic 对 Agent 的定义也强调这一点：任务明确后，Agent 会独立推进，并在执行过程中不断从工具结果、代码执行结果等环境反馈里获取“ground truth”，必要时在检查点回到人类这里确认。

更进一步说，很多你以为是“复杂 Agent”的系统，实际上可能还只是单Agent + 工具 + 若干工作流模板。Anthropic 明确列出的 prompt chaining、routing、parallelization、orchestrator-workers、evaluator-optimizer，本质上是从简单到复杂的组合模式，不必一上来就理解成“多智能体系统”。

### 1.2 多Agent是什么

多Agent不是简单地“多开几个模型实例”，而是把任务拆给多个具有独立上下文、独立职责、甚至可相互通信的执行单元。在 Anthropic 当前文档体系里，至少能看到两类典型形态：

- **Subagents**：适合“主会话派一个专门工人去做侧任务，回来交总结”，它们有独立上下文、可受限工具、可选更便宜模型，但通常只向主 Agent 汇报。
- **Agent teams**：适合“多个成员并行工作、直接沟通、共享任务列表、自主协同”，但它是实验特性，默认关闭，而且官方明确提示它有已知限制、协调开销更大、token 消耗显著高于单会话。

所以，多Agent真正的价值不在“看起来更高级”，而在于：任务本身是否天然需要并行探索、跨角色协作、相互质疑与综合。如果没有这些需求，多Agent往往只是在制造协调成本。

## 2. 单Agent的最佳实践：绝大多数项目先从这里起步

Anthropic 的建议非常克制：先直接用 LLM API 和基本组件，很多模式几行代码就能实现；即便用了框架，也要理解底层，不要把抽象层当黑盒。单Agent的核心是“聚焦、可控”，具体实践可分为三步：

## 2.1 推荐起点：增强型LLM，而非直接上多Agent

一个足够能打的单Agent，通常具备以下核心要素，能覆盖多数真实开发任务（如代码修复、需求拆解、文档生成、接口联调、轻量测试编排）：

- 一个主循环：理解任务 → 选工具 → 执行 → 读取结果 → 调整下一步
- 一组高质量工具：读文件、搜代码、运行命令、查文档、调用业务 API
- 明确停止条件：最大轮数、失败回退、人工审批点
- 基本可观测性：每一步 prompt、工具调用、输出摘要、失败原因、token/耗时统计

## 2.2 先加Skill，再考虑加Agent

如果主Agent反复在某些领域做不好，第一反应不该是“再造一个Agent”，而应该是：**先把高频知识、流程、检查清单、工具调用规范沉淀成Skill**。

Anthropic 的 Claude Code 文档对 Skill 的定位非常清楚：Skill 适合承载重复粘贴的 playbook、checklist、多步流程，且是**按需加载**的，不用时几乎不占上下文成本。比如：

- 把代码评审流程封成 skill
- 把 Vue3 页面脚手架规范封成 skill
- 把接口联调 checklist 封成 skill
- 把安全扫描/发版步骤封成 skill

这和“碰到瓶颈先加skill”的实践高度一致，也是最经济、最可控的能力增强方式。

## 2.3 单Agent不够时，先升级 workflow，而非直接上多Agent

很多人误以为“单Agent能力不足，就该上多Agent”，实则Anthropic给出的第一批升级路径，是更便宜、更可控的**workflow模式**，而非直接升级为多Agent：

- **Prompt chaining**：固定多步，适合“先写提纲，再校验，再生成正文”这类顺序任务。
- **Routing**：先分类，再选择不同后续流程，适合把简单问题路由到小模型，难题交给强模型。
- **Parallelization**：并行处理多个独立维度，适合多角度评审或多路检查。
- **Evaluator-optimizer**：生成者和评估者形成闭环，适合“有明确评价标准”的优化任务。

这些模式能在不增加拓扑复杂度的前提下，解决单Agent的多数能力瓶颈。

## 3. 多Agent的最佳实践：不是不能用，而是要“有证据地用”

多Agent不是“高级选项”，而是“单Agent和工作流都证明不够用时的升级方案”。使用时需牢记“代价与价值对等”，核心实践围绕“场景适配、成本控制”展开。

### 3.1 什么时候多Agent真有价值

Anthropic 对 agent teams 的说明很直接：它最适合 **并行探索确实能带来价值** 的任务，典型场景包括：

- 大范围代码改造：前端、后端、测试同时改
- 多假设调试：每个Agent验证一个root cause
- 多源研究：不同Agent查不同文档/仓库/接口行为
- 大型评审：架构、性能、安全、测试分别给意见

反之，对于顺序性强、同文件高冲突、依赖关系多的任务，单会话或subagent往往更合适。

### 3.2 多Agent的真实代价（官方明确提示）

- autonomous agents 本身就有更高成本和复合错误风险；
- agent teams 需要共享任务表、消息通信、协调分工，token 消耗显著更高；
- 实验特性意味着仍有会话恢复、协调、关闭行为等限制。

工程上翻译过来就是：你会额外承担上下文同步、任务分配/收敛、重复搜索、文件冲突、调试难度增加等成本。

### 3.3 多Agent最稳的落地方式（循序渐进）

不建议一步到位上agent team，最稳妥的升级顺序是：

**单 Agent → 单 Agent + Skill → 单 Agent + Workflow → 主 Agent + Subagents → Agent Team**

这个顺序和 Anthropic 的文档结构高度一致：先是增强型LLM，再到工作流，再到Agent；在 Claude Code 产品侧，也是先Skill、再Subagent、再Agent Teams。

其中，Subagents是过渡阶段的最佳选择——主Agent负责统筹，Subagents负责具体侧任务（如需求分析、UI生成、API映射、QA审查），各自有独立上下文，既解决了单Agent上下文过载问题，又避免了agent team的高协调成本。

## 4. 实用判断标准：到底该上单Agent还是多Agent

可直接套用以下逻辑，快速决策：

### 4.1 适合单Agent的情况

- 目标比较集中，主任务链条清晰；
- 主要瓶颈在知识不足、工具调用不稳、步骤容易漏；
- 任务需要一个统一上下文持续推进；
- 更在意成本、可控性、易排查性。

这时优先做：**主 Agent + 工具 + Skill + 必要工作流**

## 4.2 适合多Agent的情况

- 任务天然可以拆成几个相对独立的子问题；
- 各子问题需要不同视角或专业角色；
- 并行推进能明显缩短时间，且不会频繁争抢同一份上下文；
- 最终需要汇总、比较、交叉验证多个结论。

这时再考虑：orchestrator-workers / subagents / agent teams

## 5. “简单起步”与 Spec Kit 的互补关系（不冲突，反而是绝配）

很多人会疑惑：“强调简单起步，是不是就不需要Spec Kit了？”答案是：不冲突，二者关注的层级不同，反而能形成互补。

### 5.1 核心区别：各自解决什么问题

- 简单起步：管的是运行时架构复杂度，核心是“别一开始就搞复杂拓扑、别过度工程化”，解决“系统太复杂不好维护”的问题。
- Spec Kit：管的是开发过程和意图管理，核心是“先把what/why说清楚，再做how”，解决“需求模糊、实现跑偏”的问题。

### 5.2 为什么互补：Spec Kit 会强化“简单起步”

Spec Kit 文档本身就带有很强的“反过度工程化”倾向，其核心原则包括 **simplicity over cleverness**、**start simple, add complexity only when proven necessary**，且强调：

- specify 阶段聚焦 what/why，不提前锁死技术栈；
- plan 阶段明确技术约束，避免过度设计；
- tasks 拆成可验证小步，避免一次性生成整个功能；
- checklist 和 analyze 阶段提前发现问题，避免后期返工。

### 5.3 避免冲突：正确使用 Spec Kit 的关键

真正的冲突不是来自工具本身，而是误用：

- ❌ 错误：把Spec写成“提前锁死未来三年的复杂架构”；
- ❌ 错误：在plan阶段预埋大量“可能以后会用到”的抽象层；
- ✅ 正确：用Spec Kit把“简单起步”的原则写进constitution/plan，比如明确“MVP阶段只允许单Agent”“优先使用官方原生能力，不额外封装框架壳层”。

一句话总结：Spec Kit 应该帮助你把“为什么要简单、简单到什么程度、什么时候再升级”写进规则，而不是帮你合理化过度工程化。

## 6. 最推荐的落地姿势：Spec Kit + 简单起步 组合

---

把两者的优势结合起来，形成一套可复用的开发流程，既保证质量，又控制复杂度：

### 6.1 第1步：用 Spec Kit 明确需求与约束（先做对）

- 用 `/speckit.constitution` 写清核心原则：MVP阶段只允许单Agent、优先Skill而非多Agent、不做过度抽象；
- 用 `/speckit.specify` 明确目标、上下文、完成标准，聚焦what/why，不提前定技术栈；
- 用 `/speckit.clarify` 消除歧义，避免后期跑偏；
- 用 `/speckit.checklist` 验证需求完整性。

### 6.2 第2步：按简单起步原则搭建单Agent基础（先跑通）

- 搭建单Agent主循环，只接入最少必要工具；
- 把高频流程、规范封装成Skill，按需加载；
- 拆小任务，每完成一个就做最小验证（lint、build、简单测试）。

### 6.3 第3步：逐步升级，按需增加复杂度（再优化）

- 先升级工作流（chaining、routing等），解决单Agent的瓶颈；
- 再根据需求，逐步引入Subagents，负责独立侧任务；
- 最后，只有在并行协作价值显著时，才考虑Agent Team。

## 7. 项目示例：AI Vue3 管理后台页面生成助手（落地演示）

---

以“前端团队内部使用的AI页面生成助手”为例，演示如何结合简单起步与Spec Kit，落地单Agent到多Agent的升级。

### 7.1 项目定位（用 Spec Kit 明确需求）

核心目标：输入一段自然语言需求，自动生成Vue3管理后台页面基础实现，覆盖列表页、详情页、编辑弹窗，生成结果可预览、可编辑、可人工审核导出。

核心约束（写进constitution）：MVP阶段只允许单Agent，优先使用Vue3原生能力，不额外封装框架，生成结果需通过lint和基础测试。

## 7.2 技术栈设计（简单起步，不过度工程化）

```
1 frontend/ # Vue3前端（需求输入、结果预览、人工审核）
2 src/
3 api/、components/、composables/、router/、stores/、views/、
 utils/、types/
4 mock/
5 tests/
6
7 agent-backend/ # Node.js服务（调用模型、管理会话、执行工具）
8 src/
9 agents/、skills/、tools/、sessions/、eval/
```

前端：Vue3 + Vite + Pinia + Vue Router + Element Plus + Vitest + Playwright

Agent后端：Node.js + 模型API + 工具链（读模板、生成文件、执行校验）

## 7.3 第一阶段：单Agent MVP（先跑通）

### 7.3.1 单Agent职责

仅做一件事：把“自然语言需求”转换成“可审核的Vue3页面草案”，内部流程固定为：

```
1 接收需求 → 解析业务意图 → 提取页面类型 → 调用模板与Skill → 生成代码 → 执行校验
 → 输出草稿
```

### 7.3.2 必要工具（最少化）

- `read_component_library`：读取组件规范和业务模板
- `load_page_template`：载入对应页面模板（列表/详情/表单）
- `generate_vue_files`：生成Vue文件草稿
- `run_lint_and_test`：执行静态检查和基础测试
- `save_draft_files`：保存生成结果
- `request_human_review`：提交人工审核

### 7.3.3 核心Skill（补全能力，不拆Agent）

- `vue3-admin-patterns`：列表页布局、组件命名、Pinia使用规范等；
- `api-contract-checklist`：REST接口规范、字段映射、空状态处理等；
- `ui-review-checklist`：表单校验、三态处理、权限显隐等。

## 7.4 第二阶段：升级为“主Agent + Subagents”（按需升级）

当单Agent出现上下文过载（如同时生成多页面、多模块），或需要多视角协作时，升级为：主Agent 统筹 + 4个Subagents分工，各自独立上下文：

- `spec-analyst`：解析需求、生成页面schema，工具：Read/Grep，Skill：需求澄清、页面schema规范；

- **ui-generator**：生成Vue页面与组件，工具：Read/Write，Skill：vue3-admin-patterns；
- **api-mapper**：生成API层与类型定义，工具：Read/Write，Skill：api-contract-checklist；
- **qa-reviewer**：执行校验、生成测试建议，工具：Read/Bash，Skill：ui-review-checklist、qa-smoke-test。

运行链路：用户输入需求 → 主Agent拆任务 → 各Subagents并行执行 → 主Agent汇总结果 → 人工审核。

## 7.5 升级判断标准

只有满足以下条件，才启动第二阶段升级：

- 单Agent上下文频繁过载，生成质量下降；
- 任务拆分后，各子任务可独立推进，并行能显著缩短时间；
- 需要多视角（需求/UI/API/QA）协同，单Agent难以覆盖。

## 8. 最终结论（浓缩为3句话）

1. 单Agent不是低级方案，而是所有AI Agent项目的默认起点，核心是“清晰边界 + 小步验证”；
2. 多Agent只在“并行协作能创造额外价值”时值得上，否则只会增加成本和调试难度；
3. 简单起步与Spec Kit互补，前者控架构复杂度，后者控需求与过程质量，结合使用是最稳的落地方式。